VERSION
14.0

AMS GRA ARCHITECTURE VOLUMES

Common Infrared (IR) Multi-Function Aperture (MFA) Encapsulation Layer (MEL) Interface Description Document (IDD)

AFLCMC/WA

April 24, 2026

U.S. GOVERNMENT NOTICES: Furnished to the U.S. Government with UNLIMITED RIGHTS as defined in DFARS 252.227-7013, this takes precedence over any restrictive marking herein.

Table of Contents

1 Common IR MEL 1
1.1 Introduction . . . 1
1.2 IR MEL Decomposition . . . 2
1.3 IR MEL Functional Decomposition . . . 4
1.4 Implementation Compliance . . . 6
1.5 Deprecated Function Compliance . . . 8

2 Application Programming Interface (API) Overview 9
2.1 Imagery and Other IR MFA Data Products . . . 10
2.2 IR MFA Health and Status . . . 11
2.3 IR MFA Command and Control: States and Modes . . . 11

3 IR MEL Interface Use Cases 14
3.1 Initialize IR MEL for Use . . . 14
3.2 Configure IR MEL Channels . . . 17
3.3 Configure and Command BIT . . . 20
3.4 Receive Candidate Object . . . 23
3.5 Initialize IR MEL for Local Function Initialization and Status . . . 25
3.6 Command Single Task . . . 27
3.6.1 Task Priority and Precedence in Task Scheduling Mode . . . 31
3.7 Command Multiple Prioritized Tasks from a Single Service . . . 34
3.8 Perform Scan . . . 39
3.9 Perform Stare . . . 43
3.10 Perform Track . . . 47
3.11 Receive Detailed LOS Track . . . 51

4 IR MEL Support Functions 54
4.1 Channel Comms Test . . . 54
4.2 Instrumentation . . . 55
4.3 Navigation Report . . . 57
4.4 Keepalive . . . 58
4.5 Get Track Data . . . 58
4.6 Track Data Update . . . 59
4.7 Camera Protect . . . 60

4.8 Zeroize 61

5 IR MEL Channels 62
5.1 Command and Control 63
5.2 Health and Status 64
5.3 Scheduling 64
5.4 Instrumentation 65
5.5 Infrared Search and Track (IRST) Image 66
5.6 IRST Track 67
5.7 Stacked Image 67

A Acronyms 69

B Revision History 70

List of Figures

1.2.1 IR MEL Resource Decomposition . . . 3
1.3.1 IR MEL Enabled Functions . . . 5

2.0.1 IR MEL Pipes . . . 10
2.3.1 IR MFA States and Modes . . . 12
2.3.2 AMS GRA State Transitions Reference . . . 13

3.1.1 Initialize IR MEL for Use . . . 16
3.2.1 Configure IR MEL Channels . . . 19
3.3.1 Command and Configure BIT . . . 22
3.4.1 Receive Candidate Object . . . 25
3.5.1 Initialize IR MEL for LF Initialization and Status . . . 27
3.6.1 Command Single Task . . . 30
3.6.2 Lowest Numerical Priority Wins . . . 32
3.6.3 Equal Priority sort by Precedence . . . 33
3.7.1 Command Multiple Prioritized Tasks from a Single Service . . . 38
3.8.1 Perform Scan . . . 42
3.9.1 Perform Stare . . . 46
3.10.1 Perform Track . . . 50
3.11.1 Receive Detailed LOS Track . . . 53

4.1.1 Test Channel Communications Test Rep . . . 54
4.2.1 Get Instrumentation Data . . . 56
4.3.1 Get Navigation Report . . . 57
4.4.1 Keep Connection Alive . . . 58
4.5.1 Get Track Data . . . 59
4.6.1 Track Data Update . . . 60
4.7.1 Camera Protect . . . 61

5.0.1 IR MEL Channels . . . 62

List of Tables

1.4.1 IR MEL API Reference Table 7

B.0.1 Revision History Table 70

1.0 Common IR MEL

1.1 Introduction

The Multi-Function Aperture (MFA) Encapsulation Layer (MEL) is a critical interface within the Agile Mission Suite (AMS) Government Reference Architecture (GRA) taxonomy, which provides an Application Programming Interface (API) definition to govern the interaction between a MFA and a service in the Multi-Function Processor (MFP) layer. Two MELs are delivered with the AMS GRA Starter Kit (SK): the Common Infrared (IR) MEL and the Common Radio Frequency (RF) MEL. The MELs achieve the following business and technical objectives of the AMS GRA:

- Provide a common, abstracted interface to a broad set of digital MFA types and designs
- Enable significant advances in service portability through the application of common interfaces for interactions with the MFAs
- Enable rapid insertion of new and updated capabilities through the use of software without the need to modify the MFA
- Abstracts MFA designs from the services to promoting low coupling between the design of the MFA and the implementation of services executing in the MFP

Every AMS GRA compliant IR MFA is delivered with an IR MEL implementation, and every AMS GRA compliant RF MFA is delivered with an RF MEL implementation. For more information on the RF MEL interface see the RF MEL Interface Description Document (IDD). The MEL implementation is the MFA-specific realization of the Common IR MEL API or Common RF MEL API. The MEL implementation is logically part of the MFA. However, the MEL implementation is used by a service at runtime as a statically or dynamically linked library. AMS GRA services use an MFA's specific MEL implementation to communicate with the MFA. The interface between the MEL implementation and the rest of the MFA is implementation specific and is not prescribed by the AMS GRA.

The Common IR MEL is referred to henceforth as the IR MEL for convenience.

Note: Some messages contained in the IR MEL IDD are derived from Open Mission Systems (OMS), but this note is not intended to prescribe a specific OMS version.

1.2 IR MEL Decomposition

The IR MEL Resource Decomposition diagram, shown in **Figure 1.2.1**, contains the elements involved in the use cases for the IR MEL. The IR MEL API implementation interacts with the IR MFA through the provider specific methods. This communication is across the Avionics Backplane Bus (ABB). The Skill, composed of at least one AMS GRA service, leverages the IR MEL API to interact with the MFA. The MEL is composed of the MEL Common API which is supplemented by the Common IR MEL API to add IR specific capabilities.

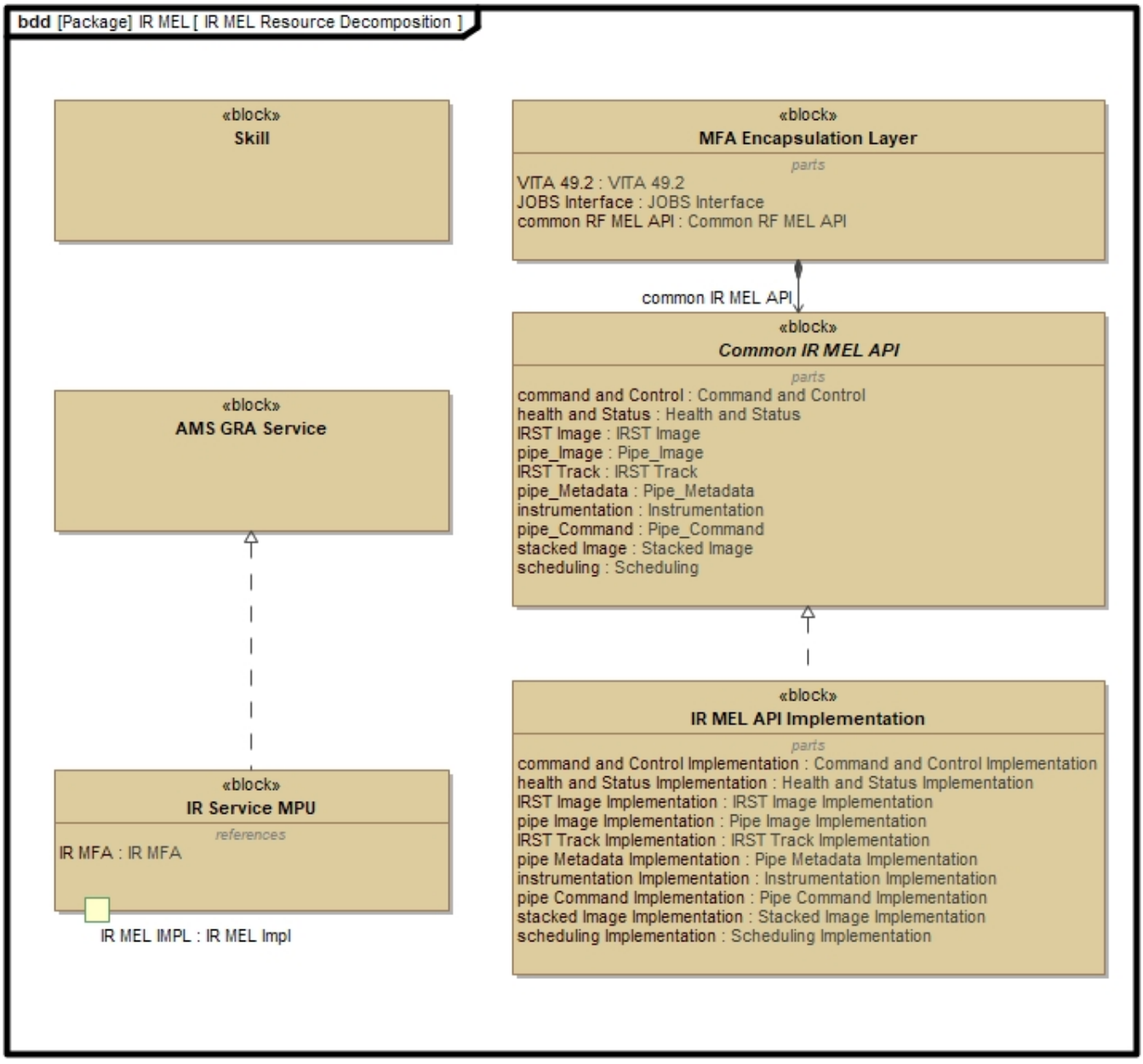


Figure 1.2.1: Individual AMS GRA Services Within a Skill Communicate With IR MFAs Over the ABB Through the IR MEL API. The IR MEL API Implementation is supplied by the MFA Provider and Enables Both Common MFA and IR MFA-Specific Capabilities.

1.3 IR MEL Functional Decomposition

The primary use cases shown in **Figure 1.3.1** show the high-level IR MFA behaviors that the IR MEL enables. They represent an AMS GRA service's goals when interacting with an IR MFA and show how the IR MEL API manifests desired capability. These use cases are demonstrative and not exhaustive in describing all possible scenarios.

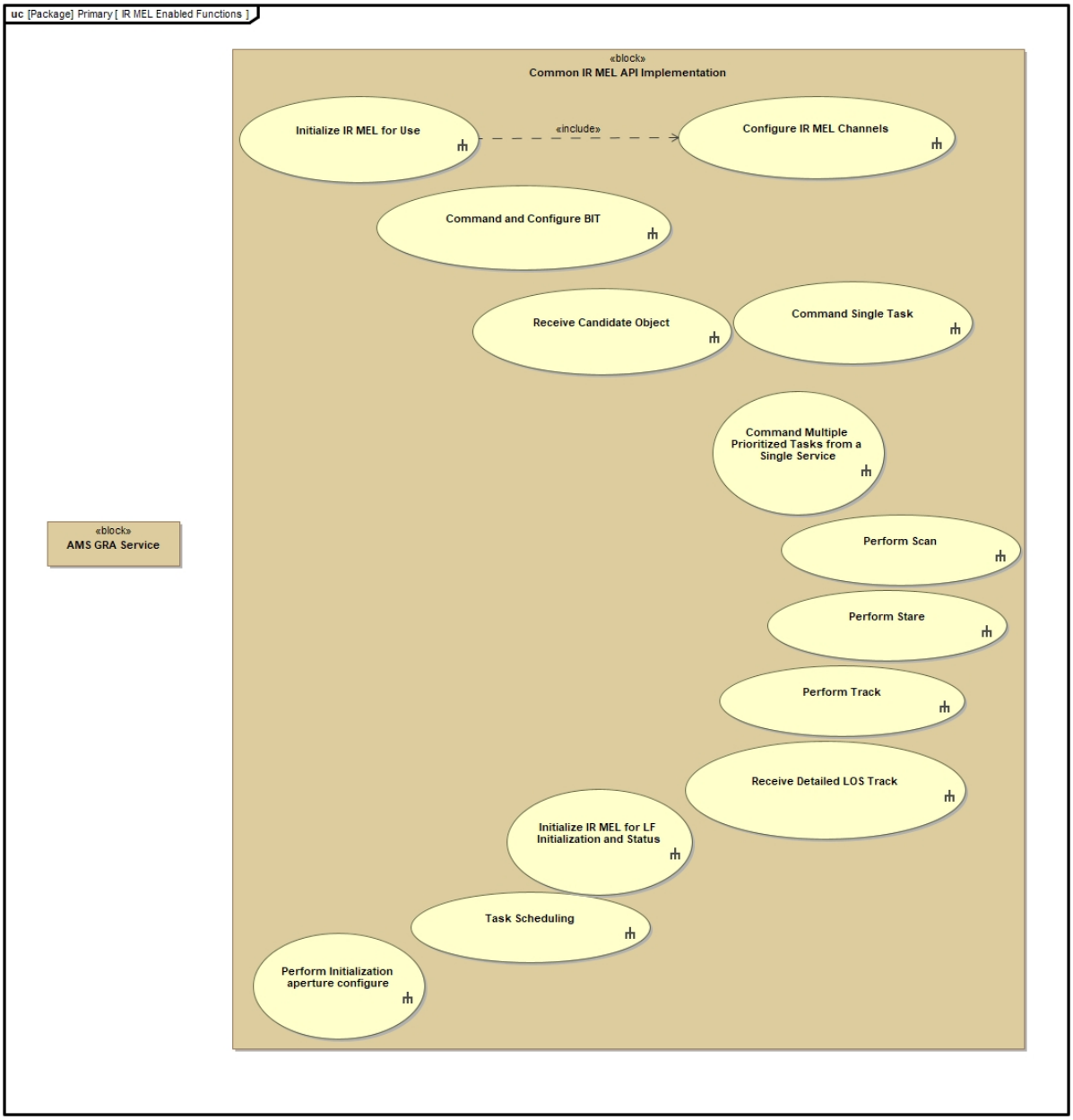


Figure 1.3.1: An AMS GRA Service Uses the IR MEL to Perform Its IR Domain Use Cases. The Most Common Use Cases are Shown Here.

1.4 Implementation Compliance

The AMS GRA SK contains C++ APIs for Common interfaces. Doxygen is an HTML output delivered with the APIs containing brief descriptions of the Common classes and methods. Within the documentation generated by Doxygen, an adopter will find prescriptive tag markings that indicate when API elements are required, conditionally required, or optional within MEL implementations. Compliant implementations must provide the corresponding content as directed by the tags. Below are descriptions of each Doxygen tag and their optionality:

- `@Required`: This element is required for all API implementations provided by a Minimum Procurable Unit (MPU).
  - When applied to a virtual function, this tag indicates that an implementation of the function must be provided in all IR MEL implementations.
  - When applied to other elements, this tag indicates that the associated data definition is used to support required MEL functions and any MEL implementation must include the element as defined in the Common API definitions.
- `@RequiredIf<capability X>`: If the corresponding capability is supported by an MPU, then the API element must be used in the provided API implementation.
  - When applied to a virtual function, this tag indicates that an implementation of the function must be provided in an IR MEL that supports the corresponding capability.
  - When applied to other elements, this tag indicates that the associated data definition is used to support MEL functions and any MEL implementation must include the element as defined in the Common API definitions if it supports the corresponding capability.
- `@Optional`: This element is optional for an MPU.
  - When applied to a virtual function, this tag indicates that an implementation of the function may be provided in an IR MEL implementation.
  - When applied to other elements, this tag indicates that the associated data definition may be used to support an IR MEL implementation as defined in the Common API definitions.

Compliant implementations of the IR MEL interface adhere to the definitions (e.g., functions, classes, macros, typedefs, and enumerations) of the Common IR MEL API. In the case of Common IR MEL API functions, compliant implementations must provide at least a minimal implementation that conforms to the function prototype defined by the Common IR MEL API. Each class, enumeration, and public member function in the Common IR MEL API is marked with a Doxygen tag stating whether it is required to be supported by all IR MEL implementations. The full list of classes, enumerations, and functions that are required, required under certain conditions, or optional are provided in the Common IR MEL Doxygen documentation. See **Table 1.4.1** for instructions on downloading the Common IR MEL Doxygen documentation. The conditions under which some API elements are required are the

following:

- **RequiredIfCameraCtrl** – The tagged API elements are required if the associated MFA provides programmable camera control.
- **RequiredIfCameraProtect** – The tagged API elements are required if the associated MFA support camera self-protection.
- **RequiredIfDetectCandidateObjects** – The tagged API elements are required if the associated MFA provides candidate object detections. Candidate objects are objects that may be targets or objects of interest, similar to a pre-detect. For more information about candidate objects, see **Section 3.4**.
- **RequiredIfImageStack** – The tagged API elements are required if the associated MFA provides Temporally Spaced Image (TSI) as an image product. For more information see **Section 2.1**.
- **RequiredIfInstrumentation** – The tagged API elements are required if the associated MFA provides instrumentation data on the instrumentation channel. For more information on instrumentation, see **Section 4.2** and **Section 5.4**.
- **RequiredIfLFSupport** – The tagged API elements are required if the associated MFA supports Local Functions.
- **RequiredIfLOSQuaternion** – The tagged API elements are required if the associated MFA provides Line of Sight (LOS) in quaternions which is available on the Infrared Search and Track (IRST) Image channel. For more information about this channel, see **Section 5.5**.
- **RequiredIfTrack** – The tagged API elements are required if the associated MFA provides tracking. For more information about tracking, see **Section 3.10** and **Section 5.6**.
- **RequiredIfTrackUpdate** – These functions are required if the associated MFA accepts track updates from a service. For more information about update tracking, see **Section 4.6**.

The referenced IR MEL API documentation detailed in **Table 1.4.1** provides further information on the API and its design.

Name	Programming Language	Description
irmel-doxygen-v.tar.gz	C++	HTML API Documentation. Extract from the SK for the selected interface, then navigate to the folders "doxygen" then "html" within the folder structure. Open the file index.html. The file name for extraction will be appended with the released version number (e.g., irmel-doxygen-v[version].tar.gz).

Table 1.4.1: IR MEL References Identify Documents that an MFA or Service Provider can Use for More Information About the IR MEL.

1.5 Deprecated Function Compliance

AMS GRA uses deprecation, denoted by the `@deprecated` tag, to identify elements that are targeted for deletion from the IR MEL API. Methods that are deprecated may continue to be used while present, but are considered optional, even if required in previous releases. Usage of these elements produces a warning at compile-time to notify adopters. Deprecated elements will be removed as a part of major release cycles. Continued use of elements after they have been deleted from the released MEL API will result in non-compliance.

2.0 Application Programming Interface (API) Overview

The IR MEL API shown in **Figure 2.0.1** outlines the software interface used by an AMS GRA Service MPU to communicate with a specific IR MFA. Services create one or more channels to a specific IR MFA, using the IR MEL to structure IR MFA interactions around functional data exchanges (channels are addressed in greater detail in Chapter 5, "IR MEL Channels"). Channels provide predefined command and metadata types, depending on the channel type, and define useful methods that facilitate and manage data exchanges across the channel which including send commands and status, listeners, and buffer management where appropriate.

These functional data exchanges include (1) Command and Control (C2) to the MFA that supports basic IR MFA control (e.g. configuration, Built-In Test, state, and test commands given by the MEL or supported by OEM); (2) MFA Health and Status (primarily the conveyance of status metadata such as Built-In Test (BIT), MFA, and local function status as well as command status) from the MFA; (3) Scheduling (tasking of mission activities and associated status) to the MFA; (4) support for the delivery of IRST Imagery and other IR MFA data products from the MFA; and (5) support for the IR MFA Instrumentation from the MFA.

The MFA MPU Volume contains brief descriptions of the IR MFA fundamental data types. The data format for most of the defined types is prescribed by the Common IR MEL API. Details can be found in the C++ header files or the associated documentation generation by Doxygen. The data format of a single IR frame is captured in the specification of the `FrameHeader` class in the Common IR MEL API. `FrameHeader` is passed as a metadata call in the API, and may be used to locate images within the buffer via the virtual function `getImageAddress`, which gets the memory pointer for the image data. The data format of a TSI is captured in the specification of the `ImageStackHeader` in the Common IR MEL. The `ImageStackHeader` is passed in through the `onImageStack` method and is used to process the TSI data within the buffer.

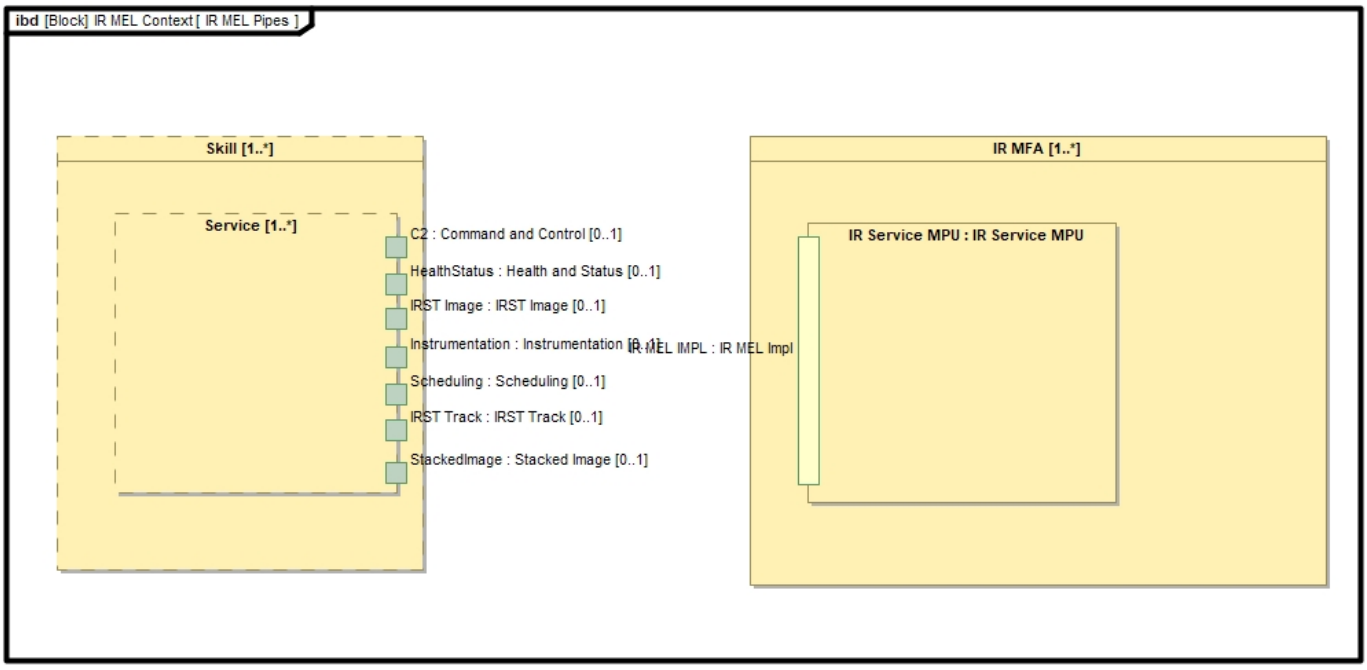


Figure 2.0.1: Services Communicate With the IR MFA Through the IR MEL API Using Channels. Channels are Made Up of an Image Pipe for Large Data Items, a Metadata Pipe for Small Data, and a Command Pipe for Commands From Service to MFA.

2.1 Imagery and Other IR MFA Data Products

The IR MEL API provides buffer management functions to facilitate the storage and access of large data over the image "pipe." Where data streams are "bufferable," the service is responsible for allocating and enabling these buffers using the channel-class-provided registration and enabling (as well as corresponding inverse) methods.

A Temporally Spaced Image (TSI) object is a set (stack) of spatially and temporally spaced images where the Δt between image frames is optimized to maximize target detection and measurement fidelity. The TSI object includes metadata for both the overall image stack as well as individual image frames. Stacked images are used for IR detection tasks, such as IRST Scan, Stare, and Track.

The Service is also responsible for implementing the listener class with functions that are called by the IR MEL, which allow the service to receive new data over the MEL API, for metadata and imagery.

2.2 IR MFA Health and Status

An IR MEL implementation provides the `MFA_Status` metadata periodically. The `MFA_Status` structure contains a list of the MFA's components and the status for each. The metadata structure is modeled after the OMS/Universal Command and Control Interface (UCI) `SubsystemStatus` message. As such, a service that uses the IR MEL can leverage the `MFA_Status` structure to populate the OMS/UCI `SubsystemStatus` message and thereby communicate the `MFA_Status` to the Mission Processing Subsystem (MPS) through the OMS Abstract Service Bus (ASB).

Additionally, an IR MEL implementation provides the `MFA_SecurityAuditRecord` metadata upon detection of a security event. The `MFA_SecurityAuditRecord` is modeled after the OMS/UCI `SecurityAuditRecord` and the `MFA_SecurityAuditRecord` structure contains a set of security event information/details including `securityEventID`, `eventTimeStamp`, `subsystemID`, outcome, and severity. Similarly to the `SubsystemStatus` message, a service that uses the IR MEL can leverage the `MFA_SecurityAuditRecord` structure to populate the OMS/UCI `SecurityAuditRecord` message and thereby communicate the `MFA_SecurityAuditRecord` to the MPS through the OMS ASB.[1]

2.3 IR MFA Command and Control: States and Modes

The states of IR MFAs align with OMS states. **Figure 2.3.1** shows one example of an IR MFA state diagram without Shutdown and Degraded states for simplicity. The embedded chart displays allowed states and state-to-state transitions.

The IR MFAs may support multiple modes within the Operate state. Within this state, there are multiple scheduling modes shown as substates in the state machine diagram. These scheduling modes include Task Scheduling, Scan Volume Scheduling, and Scan Bar Scheduling. See **Section 3.6.1** for more information about Task Scheduling.

[1] While the `MFA_SecurityAuditRecord` is available for implementers to use it is not required for a given MFA implementation to use, or for an MFA implementation to support all of the checks.

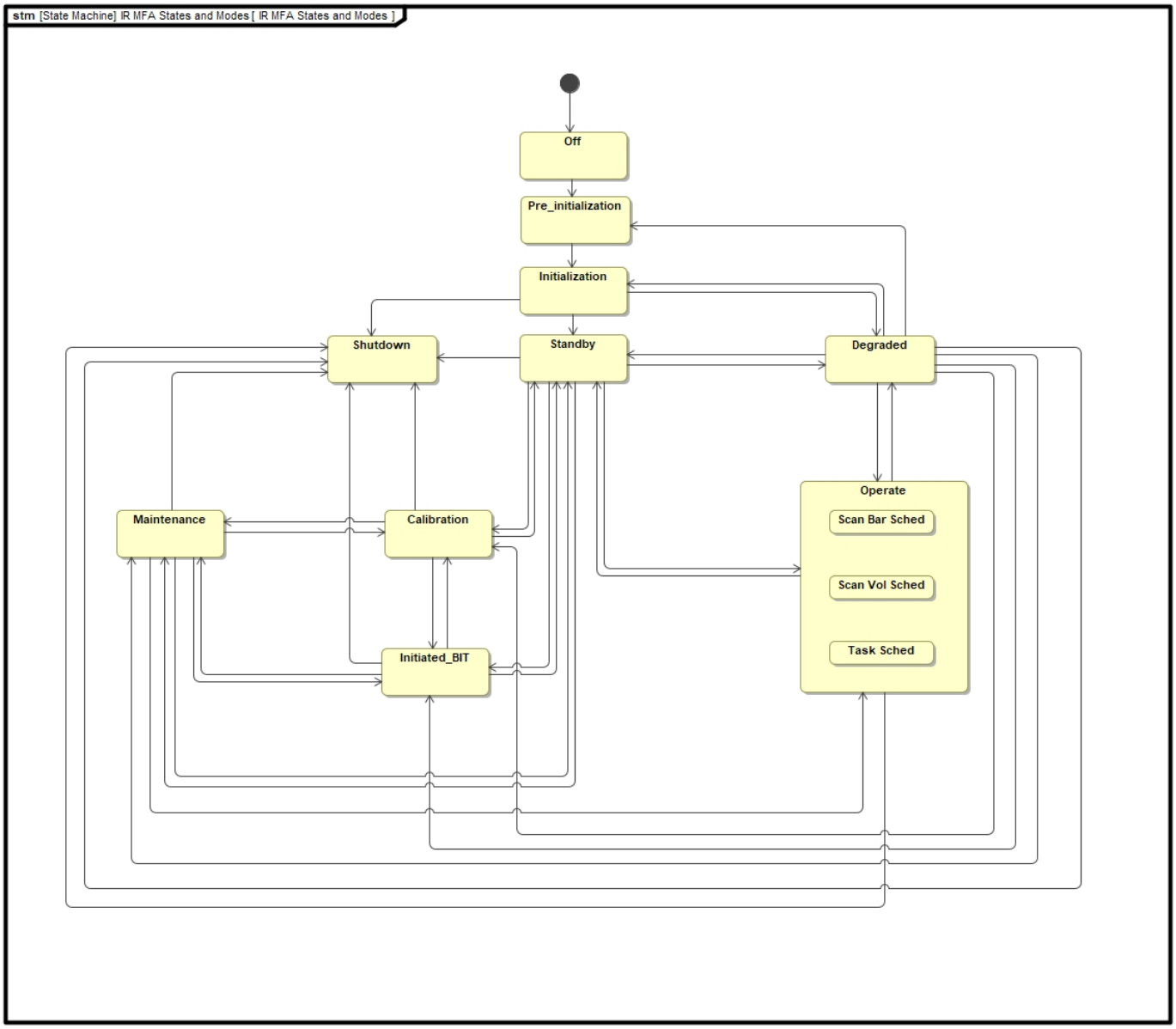


Figure 2.3.1: An IR MFA May Transition Through Several States Over the Lifecycle of the System, Which May be Communicated Back to Services Through the IR MEL.

Figure 2.3.2 displays what state transitions are allowed (yellow boxes) versus not allowed (blue boxes) in the MFA States and Modes diagram. It is not a requirement to support every state or state-to-state transition in the table but an IR MFA uses only transitions allowed in the table. Note that Shutdown is a terminal state – no transition out of Shutdown is allowed without cycling power.

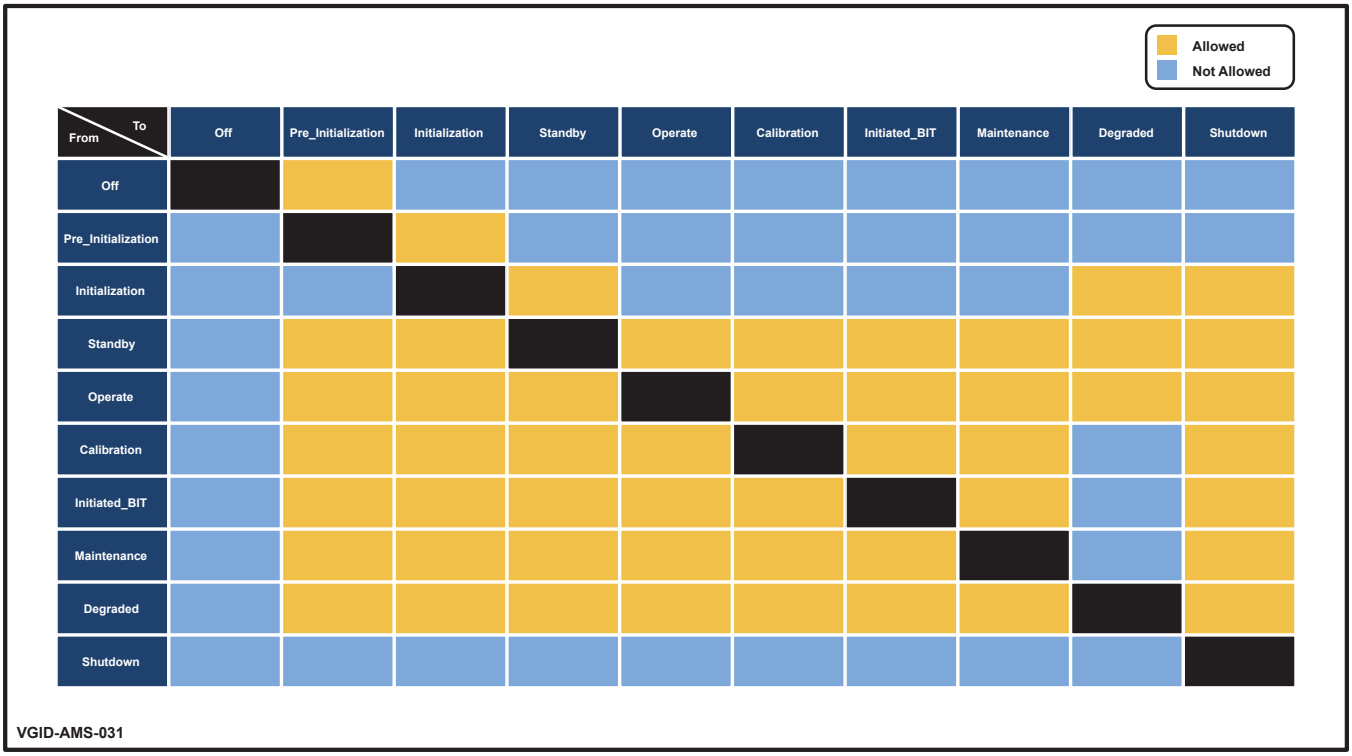


Figure 2.3.2: IR MFAs May Only Transition to Certain States Based on the Current State.

3.0 IR MEL Interface Use Cases

3.1 Initialize IR MEL for Use

Description

The sequence diagram for this use case, shown in **Figure 3.1.1**, captures how the IR MEL is initialized for use by the service.

Preconditions

Before IR MEL initialization can be started, the following conditions must be true:

- The service has successfully loaded the IR MEL library.
- The MFA is powered on and in the Pre-initialization state.
- The service will identify which IR MFAs it may employ. While these may be foreordained for a given system, the service typically has access to Mission Data and may engage in discovery processing in order to determine which MFA resources are suitable to the service's function. (The aperture is identified by a variety of descriptors, e.g., its OMS Universally Unique Identifier (UUID), its installation, or its reference string.)
- Furthermore, the service will compose its function using features provided through the IR MEL and has determined which IR MEL components, or channels, support its function.
- The IR MEL library contains the IR MEL ChannelFactory class with methods corresponding to each IR MEL channel type. The createChannel method constructs a corresponding channel associated with a specific MFA and returns a reference for the instantiated IR MEL component with appropriate built-in functions as configured.
- This use case details initialization of a single IR MEL channel for a single IR MFA. If the service desires multiple channels or multiple MFAs, the use case may be iterated upon.

Postconditions

After the IR MEL has been initialized, the following condition should be true:

- The selected IR MEL channel has been constructed by the service and its reference point is known.
- The service can send and receive data on the attached channels through the IR MEL API component's reference.

Assumptions

This use case details initialization of a single IR MEL channel for a single IR MFA. If the service desires multiple channels or multiple MFAs, the use case may be iterated upon.

Actors

The actors in the Initialize IR MEL for Use use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.1.1 shows the initialization of the IR MEL:

1. The service constructs a channel configuration object associated with the selected MFA with the intent to instantiate the selected IR MEL channel.
2. The service uses the IR MEL ChannelFactory class to construct the selected IR MEL channel by passing the channel configuration object to the appropriate IR MEL Channel Factory "load" class method corresponding to the selected IR MEL channel.
3. The IR MEL ChannelFactory constructor returns a reference for the instantiated IR MEL channel component.

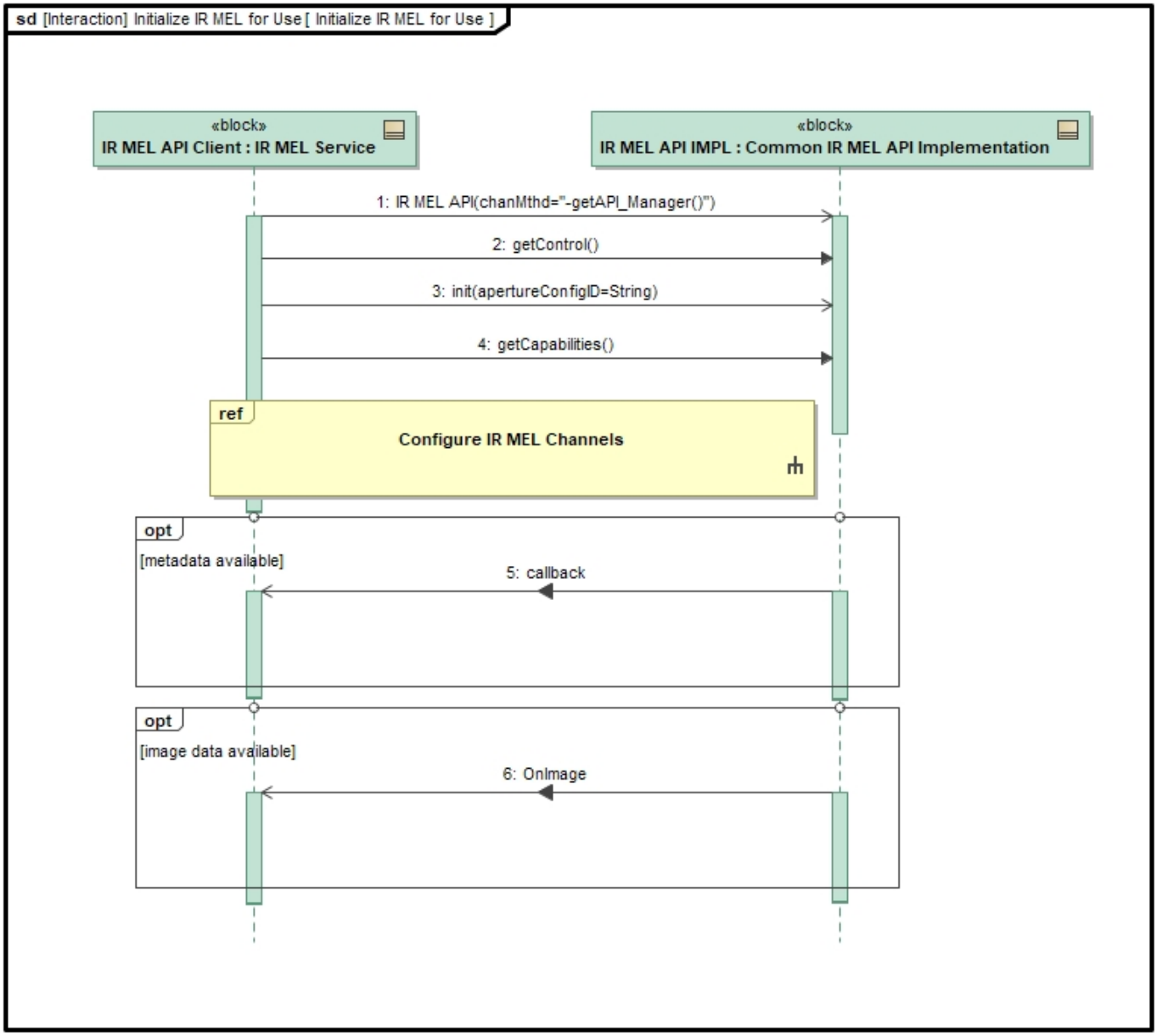


Figure 3.1.1: A Service Performs Several Actions Before Configuring IR MEL Channels. When These are Complete, the Service May Receive Metadata and Image Data.

3.2 Configure IR MEL Channels

Description

The sequence diagram for this use case, shown in **Figure 3.2.1**, is closely aligned to the Initialize IR MEL for Use use case. Whereas that use case provides a reference to a newly constructed channel object, Configure IR MEL Channels configures those channels, using the reference to gather data on the MFA's capabilities and enabling the channel to realize the service's function via the IR MEL API.

Preconditions

Before the IR MEL channels can be configured to capture required data, the following conditions must be true:

- The selected IR MEL channel has been constructed by the service according to the Initialize IR MEL for Use use case and its channel reference is known.
- The service has determined which listeners are necessary to implement the service's function.
- If the service requires access to metadata available on the channel, the service will implement functions from the defined listener classes to receive and process the data.
- If the service requires access to IRST image data, it implements functions from the channel's Image listener class when called by the IR MEL to receive and process the data. (The Image listener class comes with a buffer and buffer management methods that the service may invoke.)
- The MFA is in the Standby state.

Postconditions

After the IR MEL channels have been configured and enabled, the following condition should be true:

- The referenced IR MEL channel is enabled.
- The service can command and control the IR MFA using the referenced IR MEL channel as configured by the service.
- The service is able to receive data from the IR MFA using the referenced IR MEL channel as configured by the service.

Assumptions

There are no further assumptions to execute the Configure IR MEL Channels use case.

Actors

The actors in the Configure IR MEL Channels use case are:

- The service
- IR MEL API implementation

Flow of Events

Figure 3.2.1 shows how the IR MEL channel would be configured; the use case is recursed for each channel the service needs to configure:

1. The service creates the image listener (as required).
2. The service may request the capabilities of the associated MFA using the getCapabilities method per the channel reference [obtained via "Initialize IR MEL for Use]." (The capabilities are tailored to the channel type.)
3. Using the capabilityFlags provided in the response, the service affirms that the MFA meets capability or data requirements to configure the referenced channel.
4. If the service requires access to IRST image data, the service uses the registerBuffer method to attach the buffer to the referenced channel.
5. The service enables the referenced channel.

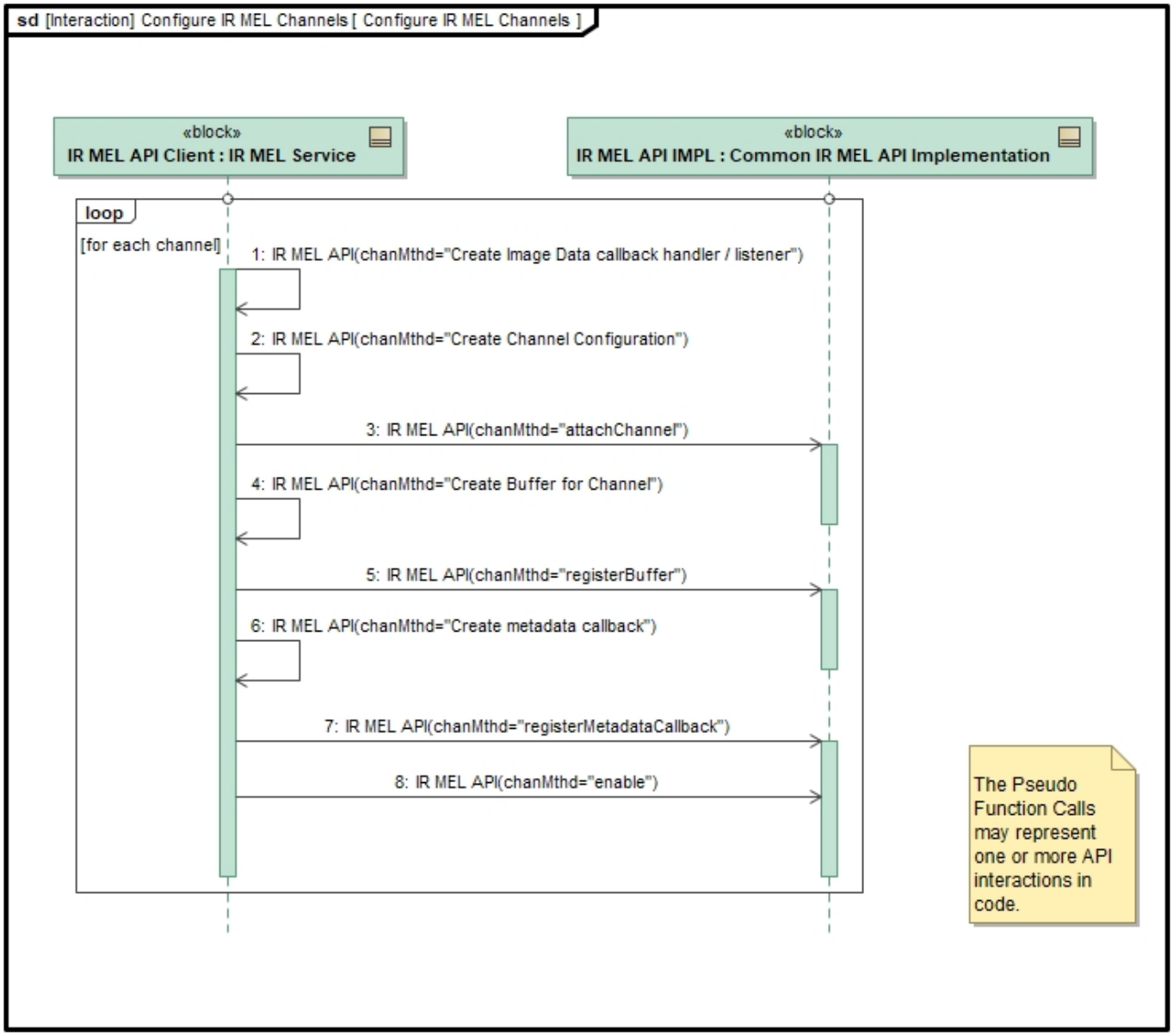


Figure 3.2.1: A Service Performs Several Actions to Configure an IR MEL Channel. It Repeats These Actions for Each Type of Channel That it Uses.

3.3 Configure and Command BIT

Description

The sequence diagram for this use case, shown in **Figure 3.3.1**, captures the exchange of BIT configuration, command, and status messages between the IR MEL and the service.

Preconditions

Before the BIT configuration and command messages can be captured between the IR MEL and service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning a reference for the Health and Status channel.
- The Health and Status channel has been set up using the Configure IR MEL Channels use case, defining send command methods [to provide BIT_Configuration data, command BIT, e.g.] resulting in the instantiation of an attached metadata listener for health and status data (including BIT status and MFA status) from the IR MEL.
- The Command and Control (C2) [Command and Control] channel has been set up using the Configure IR MEL Channels use case, defining send command methods [to provide BIT_Configuration data, command BIT, e.g.] resulting in the instantiation of an attached metadata listener for health and status data (including BIT status and MFA status) from the IR MEL.

Postconditions

After the Configure and Command BIT use case is complete, the following condition should be true:

- The MFA is ready to respond to additional commands.

Assumptions

There are no further assumptions to execute the Configure and Command BIT use case.

Actors

The actors in the Configure and Command BIT use case are:

- The service

- MEL API implementation

Flow of Events

Figure 3.3.1 shows how the BIT configuration and command messages are exchanged between the IR MEL and the service:

1. The IR MEL provides the defined `BIT_Configuration` for all associated MFA components, and the service processes the configuration data with the HealthStatus metadata listener. (The service uses the BIT_Configuration data to inform its status reporting.)
2. The IR MEL provides `MFA_Status` over the HealthStatus channel according to its configuration and is processed by the service using the metadata listener.
3. The service commands the IR MEL to perform BIT via a send command method across the C2 channel.
4. The IR MEL reports that it has received the BIT command through `CommandStatus`, which is detected by the metadata listener for the C2 channel.
5. The IR MEL provides BIT status over the HealthStatus channel according to its configuration and is processed by the service using the associated metadata listener.

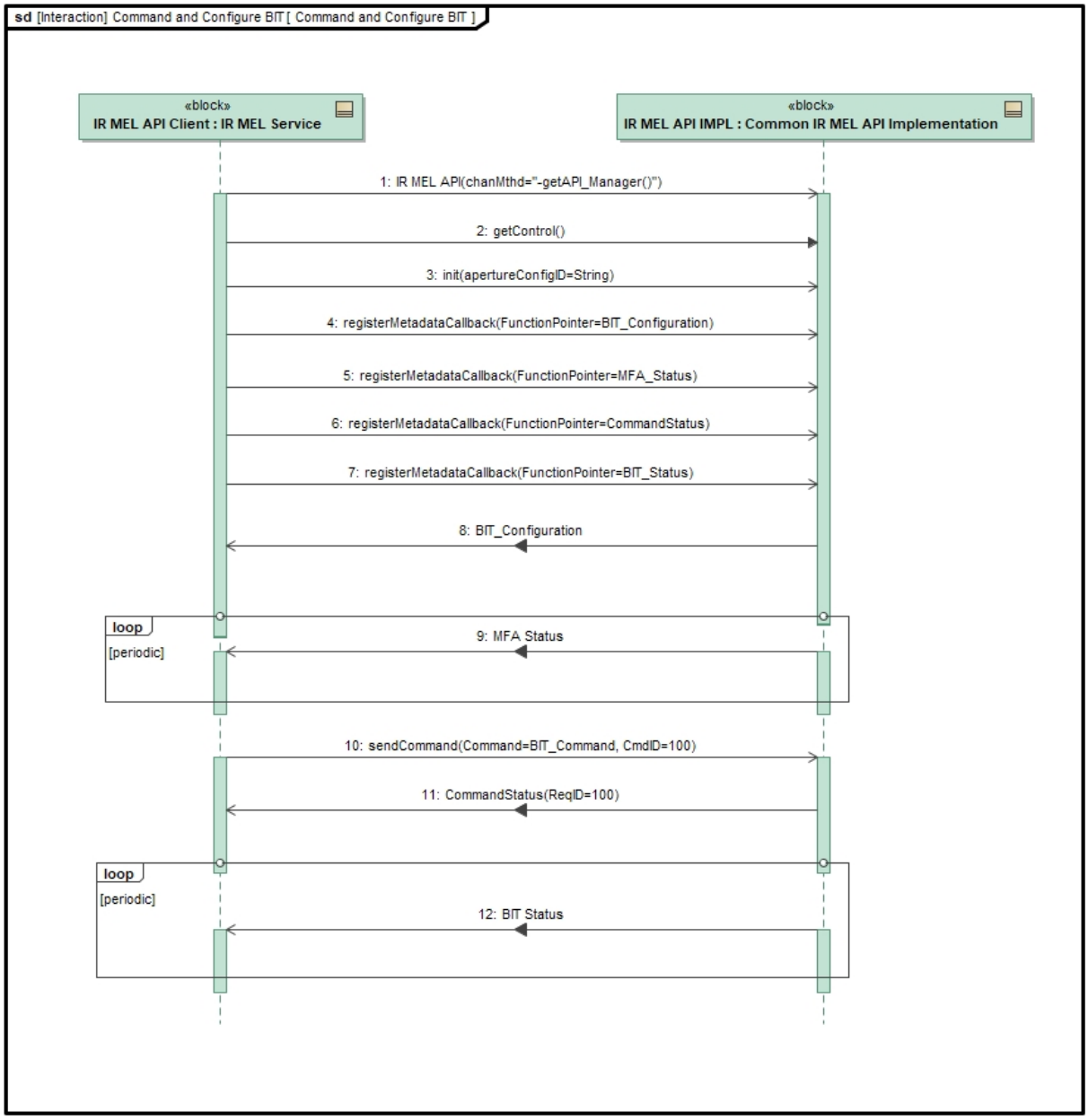


Figure 3.3.1: A Service May Configure BIT by Initializing and Registering Metadata Callbacks for BIT-related Messages. A Service May Then Send a BIT Command to Receive BIT Status.

3.4 Receive Candidate Object

Description

A candidate object is an object that the MFA has detected in the image frame and reports to the service as a potential target to continue observing or tracking. The candidate object metadata provided through the IR MEL includes the IRST Line of Sight Coordinate Correction (LCC) kinematics of the object in the image frame, the platform kinematics, and the features of that object. Optionally, the `CandidateObject` structure can be replaced with the `CandidateObjectPreProc` structure to provide additional information preprocessed internal to the MFA. As shown in **Figure 3.4.1**, the IR MEL provides candidate object metadata through the metadata pipe using the `CandidateObjectMessage`. The `CandidateObjectMessage` structure contains a list of the candidate objects that were found from one or more image frames. An IR MEL implementation provides `CandidateObjectMessage`s as they are generated by the MFA. There is not a command required from the service to prompt the generation of this metadata.

Preconditions

Before the MFA can detect the candidate object in the image frame, and report it to the service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning a reference for the IRST Image channel.
- The IRST Image channel has been been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for candidate objects from the IR MEL. .
- The MFA has been commanded into the Operate state.

Postconditions

After the MFA has detected the candidate object in the image frame, and reported it to the service, the following condition should be true:

- The MFA is ready to perform another task.

Assumptions

The following functions and conditions are assumed true for the candidate object to be properly reported to the service:

- The MFA supports `CandidateObjData` tasks.
- The service can confirm this capability of the IR MEL by calling the `getCapabilities` function after initializing the IR MEL.

Actors

The actors in the Receive Candidate Object use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.4.1 shows the MFA reporting a detected object to the service:

1. An MFA detection is provided to the IR MEL.
2. The service receives and processes `CandidateObjectMessage`s from the IR MEL via the metadata listener on the Image channel.

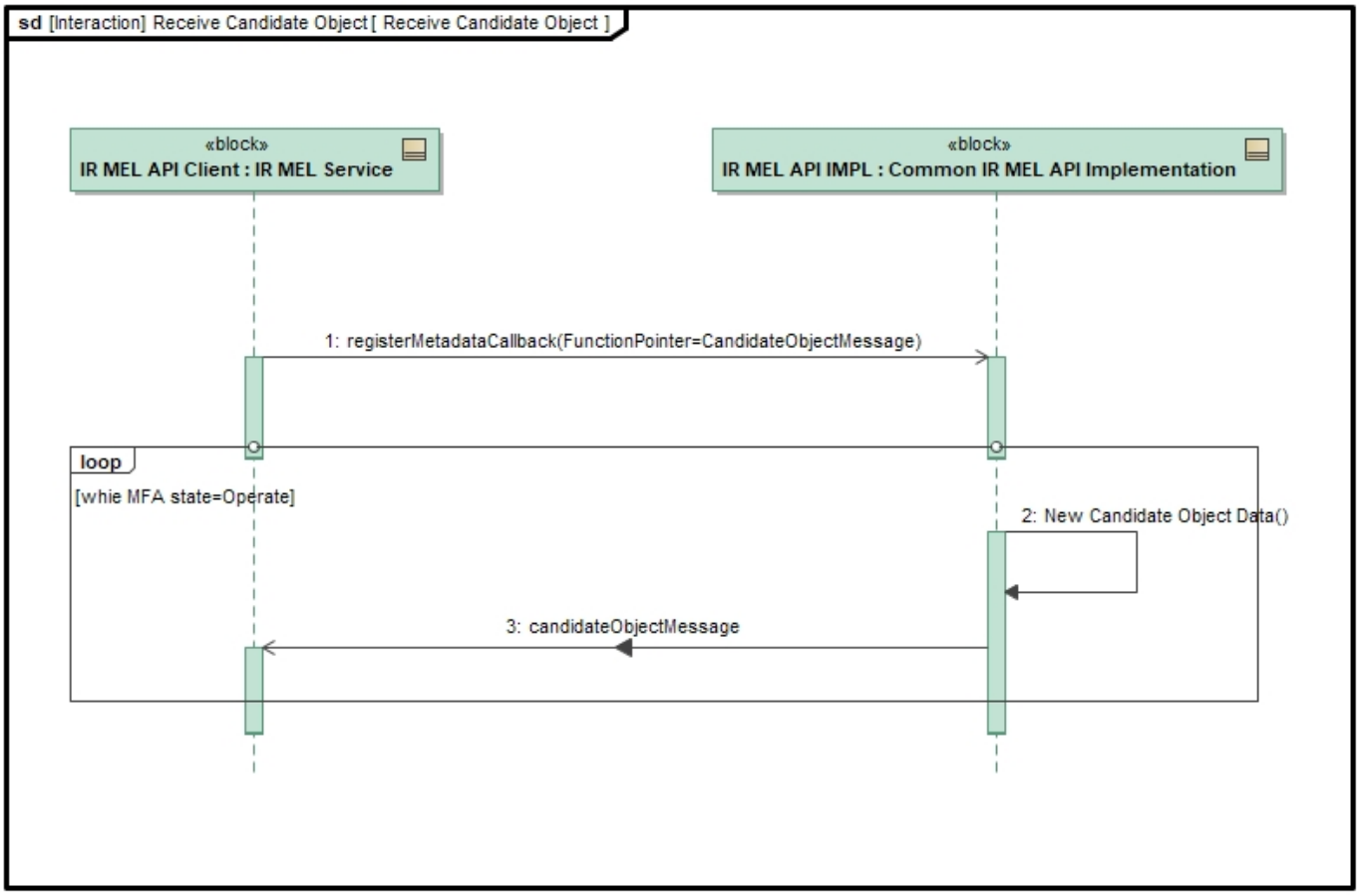


Figure 3.4.1: An IR MFA May Report Candidate Objects to a Service Through a Metadata Callback.

3.5 Initialize IR MEL for Local Function Initialization and Status

Description

An `LFStatus` is an object that captures the status of a Local Function to report to the service. The `LFStatus` consists of the `apertureConfigID`, `LFID`, `LFStatus`, and fault values. The `apertureConfigID` denotes the MFA configuration identification string. The `LFID` captures the unique Local Function Identification (ID). The `LFStatus` captures an enumerated value which could be `Initializing`, `Normal`, or `Degraded`. The fault value represents an enumerator which could be `None`, `Caution`, `Warning`, or `Failed`. As shown in **Figure 3.5.1**,

the IR MEL provides local function status metadata through the metadata pipe using the `LFStatus`. An IR MEL implementation provides the `LFStatus` as they are generated by the MFA. There is not a command required from the service to prompt the generation of this metadata.

Preconditions

Before the IR MEL can provide the local function status metadata, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning a reference for the Health and Status channel.
- The Health and Status channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for LFStatus from the IR MEL.
- The IR MEL has been initialized with a configuration using local functions.

Postconditions

None.

Assumptions

There are no further assumptions to Initialize IR MEL for Local Function Initialization and Status.

Actors

The actors in the Initialize IR MEL for Local Function Initialization and Status use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.5.1 shows the local function being initialized and sent to the service:

1. At a periodic interval, the MFA gathers the current local function status.
2. The service receives and processes `LFStatus` from the IR MEL via the metadata listener on the HealthStatus channel.

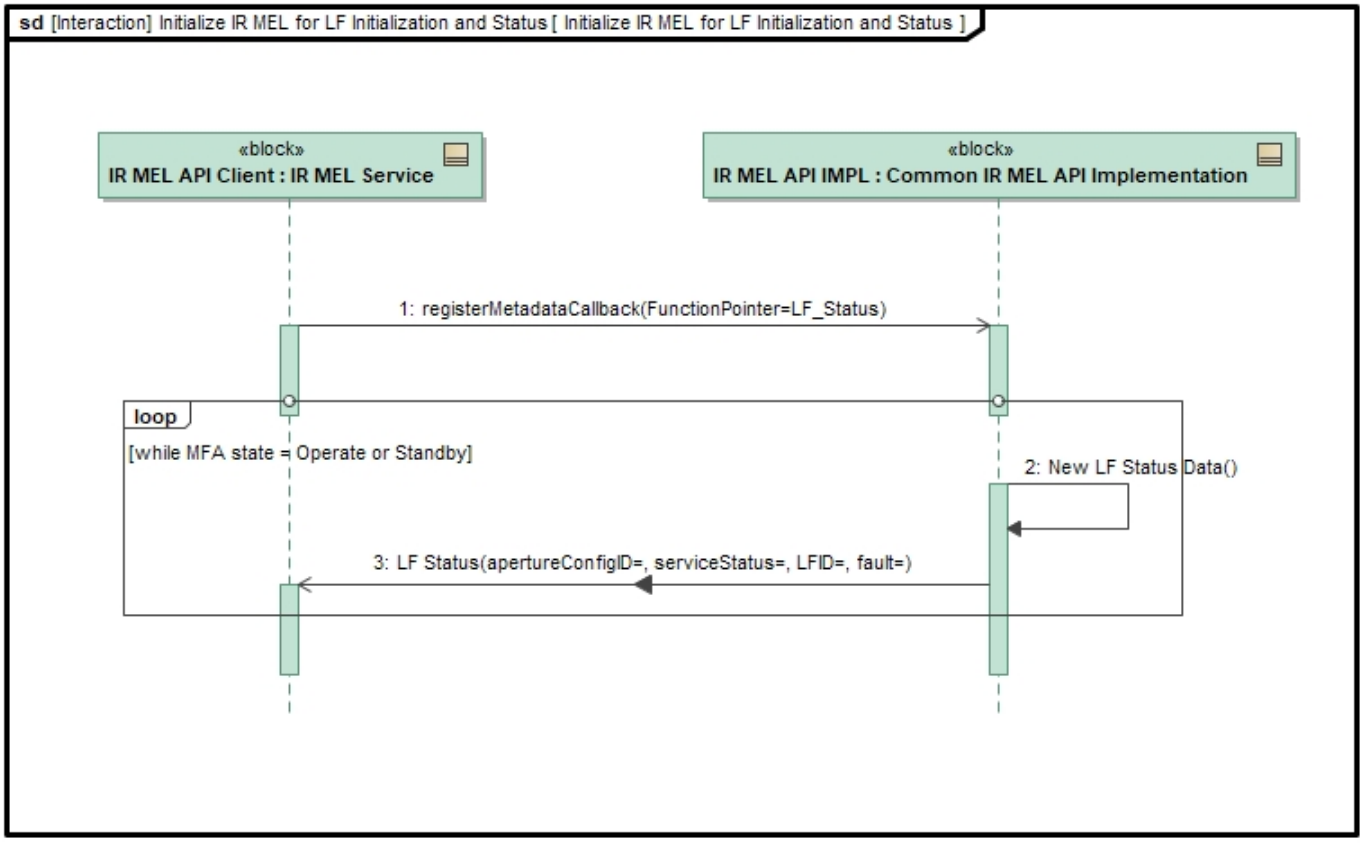


Figure 3.5.1: An IR MFA Can Report Local Function Status to a Service Through a Metadata Callback.

3.6 Command Single Task

Description

The sequence diagram for this use case, shown in **Figure 3.6.1**, represents how IR MFAs support task scheduling. The simplest task scheduling paradigm consists of a service commanding the MFA to perform a single task using the Execute Task command. The specific tasks that can be commanded are described in later sections of this document and include IRST scan (**Section 3.8**) and stare (**Section 3.9**).

Preconditions

Before the Command Single Task use case can support task scheduling, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning references for the C2 and Scheduling channels.
- The C2 channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `CommandStatus` from the IR MEL.
- The Scheduling channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listeners for `ExecuteTaskAck` and `TaskExecutingRep` from the IR MEL.
- The MFA has been commanded into the Operate state.

Postconditions

After the MFA has performed the task, the following condition should be true:

- The IR MEL is ready to receive a new task.

Assumptions

The following functions and conditions are assumed true for the task to be properly completed:

- The service affirms that the MFA is able to perform the task by requesting relevant capabilities via the IR MEL Scheduling channel [using its `getCapabilities` method].

Actors

The actors in the Command Single Task and Status use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.6.1 shows the service commanding the MFA to perform a task:

1. The service (shown as one service in this diagram, but could be a collection of services) uses the Scheduler

channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as any relevant parameters, to convey the desired task to the IR MEL.

2. As with any command sent to the IR MEL via `sendCommand`, the MFA provides a `CommandStatus` [over the C2 channel] in response.
3. The IR MEL acknowledges the command with an `ExecuteTaskAck` [over the Scheduling channel].
4. The MFA indicates it is executing the task via the IR MEL by providing one or more `TaskExecutingRep` messages to the service, over the Scheduling channel, using the attached metadata pipe.
5. The MFA completes the task.
6. After the task is complete, the IR MEL sends a `TaskExecutingRep` message indicating that the MFA has completed the task and has transitioned to idle, ready for additional tasks to be commanded.

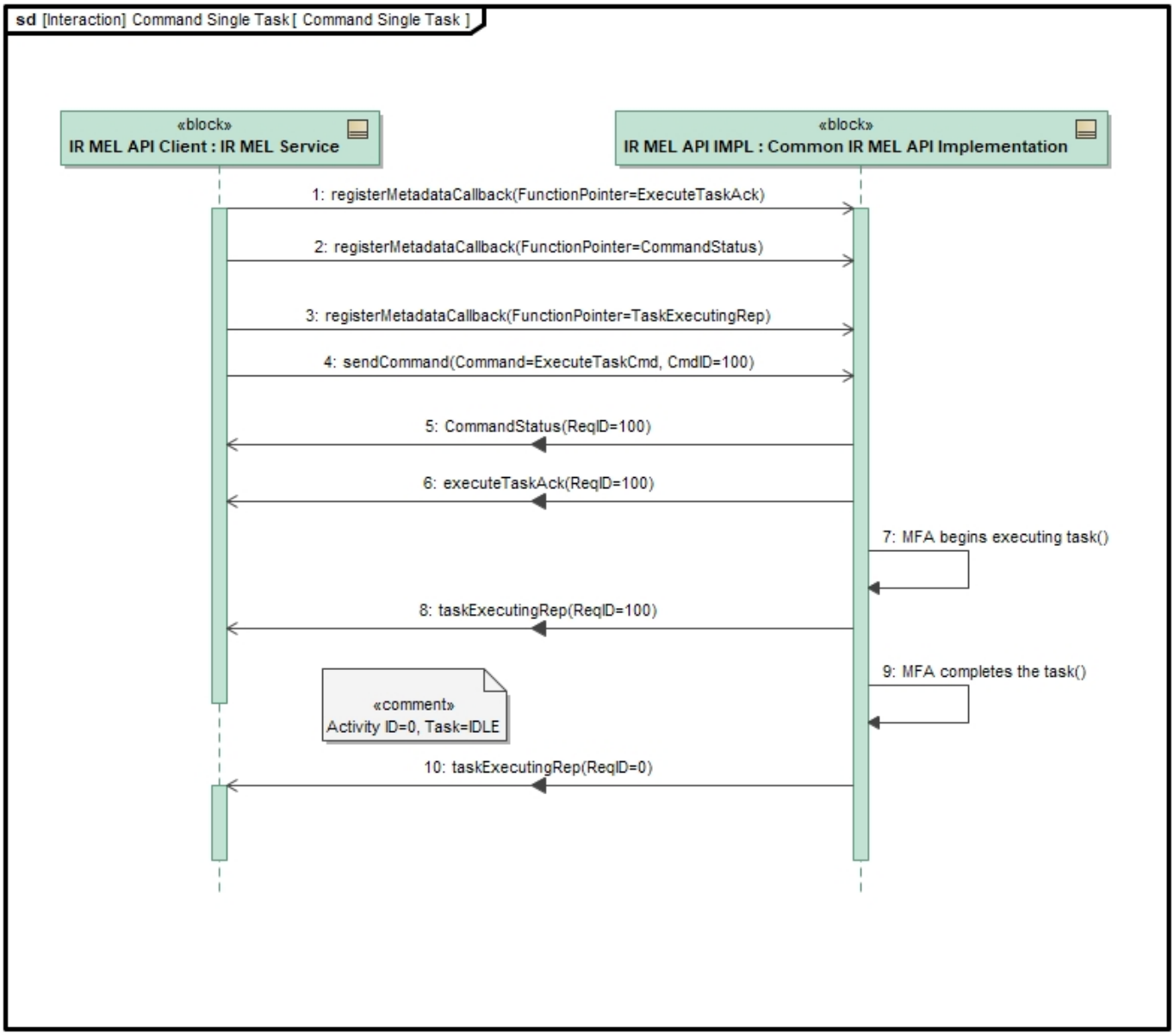


Figure 3.6.1: A Service Can Command a Task and Receive Updates on Task Status Through Metadata Callbacks. These Updates Occur During Task Execution and After Completion.

3.6.1 Task Priority and Precedence in Task Scheduling Mode

In Task Scheduling Mode, the following rules will be applied to determine a requested task's position on the task list:

Lowest Numerical Priority

When an IR MFA receives multiple tasks, a task with a lower numerical priority will rank higher in the task list than a task with a higher numerical priority. Ranking higher in the task list means the task will be considered for scheduling before a task lower on the list. For example, in **Figure 3.6.2**, on line 4, `CmdID` 200 jumps ahead of `CmdID` 100 in the task list, because `CmdID` 200 has a priority of 10 versus `CmdID` 100's priority of 100.

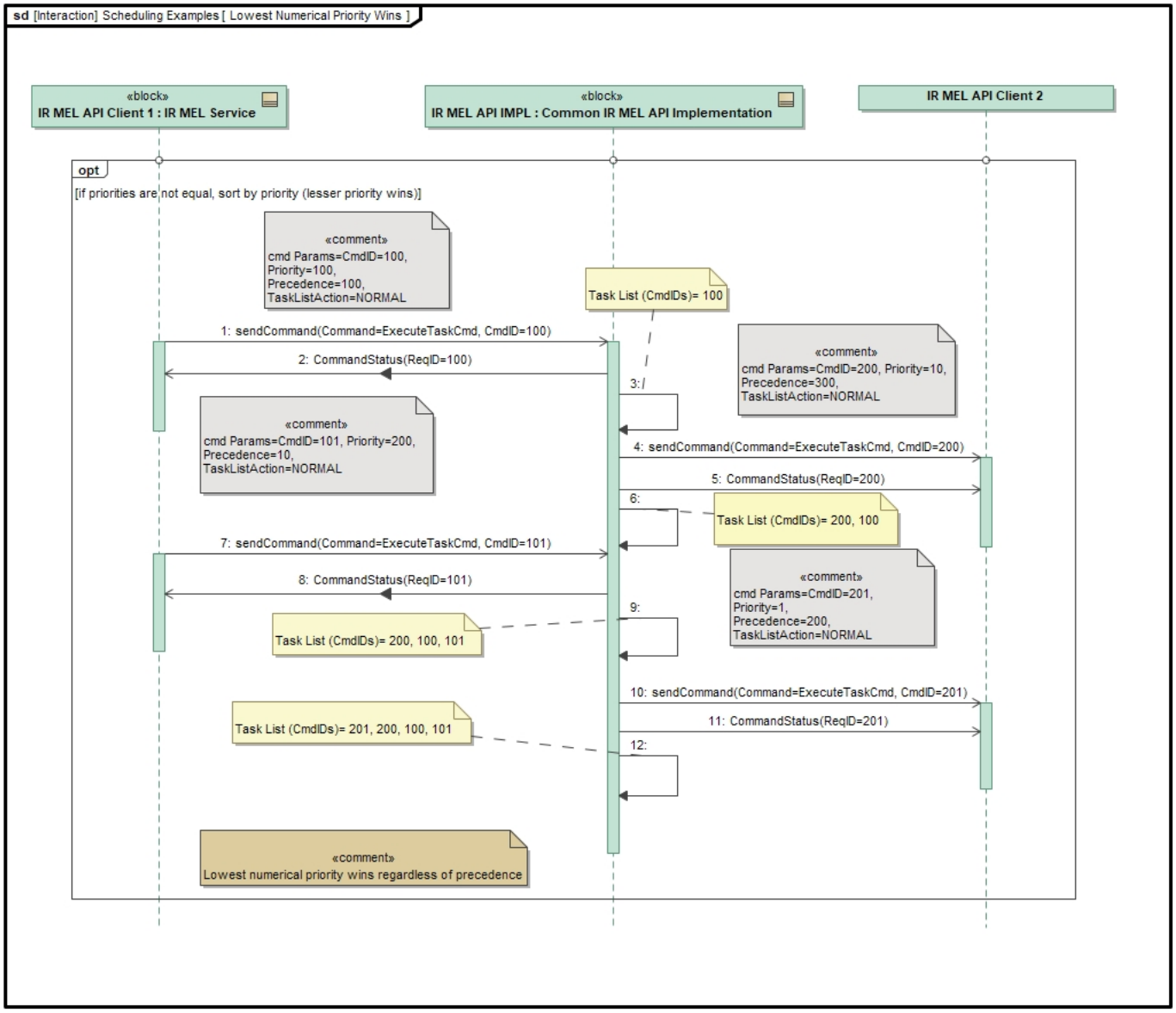


Figure 3.6.2: When Services Command Multiple Tasks, the Task With the Lowest Numerical Priority Will be Considered for Scheduling First. This Allows for Prioritization of Tasks by the Scheduler.

Equal Priority

When an IR MFA receives multiple tasks with equal numerical priorities, the task with lower numbered precedence will rank higher in the task list than a task with a higher numbered precedence. Ranking higher in the task list means that the lower numbered precedence task will be considered for scheduling before the task with higher numbered precedence (and equal priority). For example, at message 4 in **Figure 3.6.3**, `CmdID` 100 ranks higher than `CmdID` 101 in the task list. This ranking is because `CmdID` 100 has a precedence of 100 and a priority of 200 versus `CmdID` 101 that has a precedence of 200 and a priority of 200. Thus, `CmdID` 100 is considered first for scheduling.

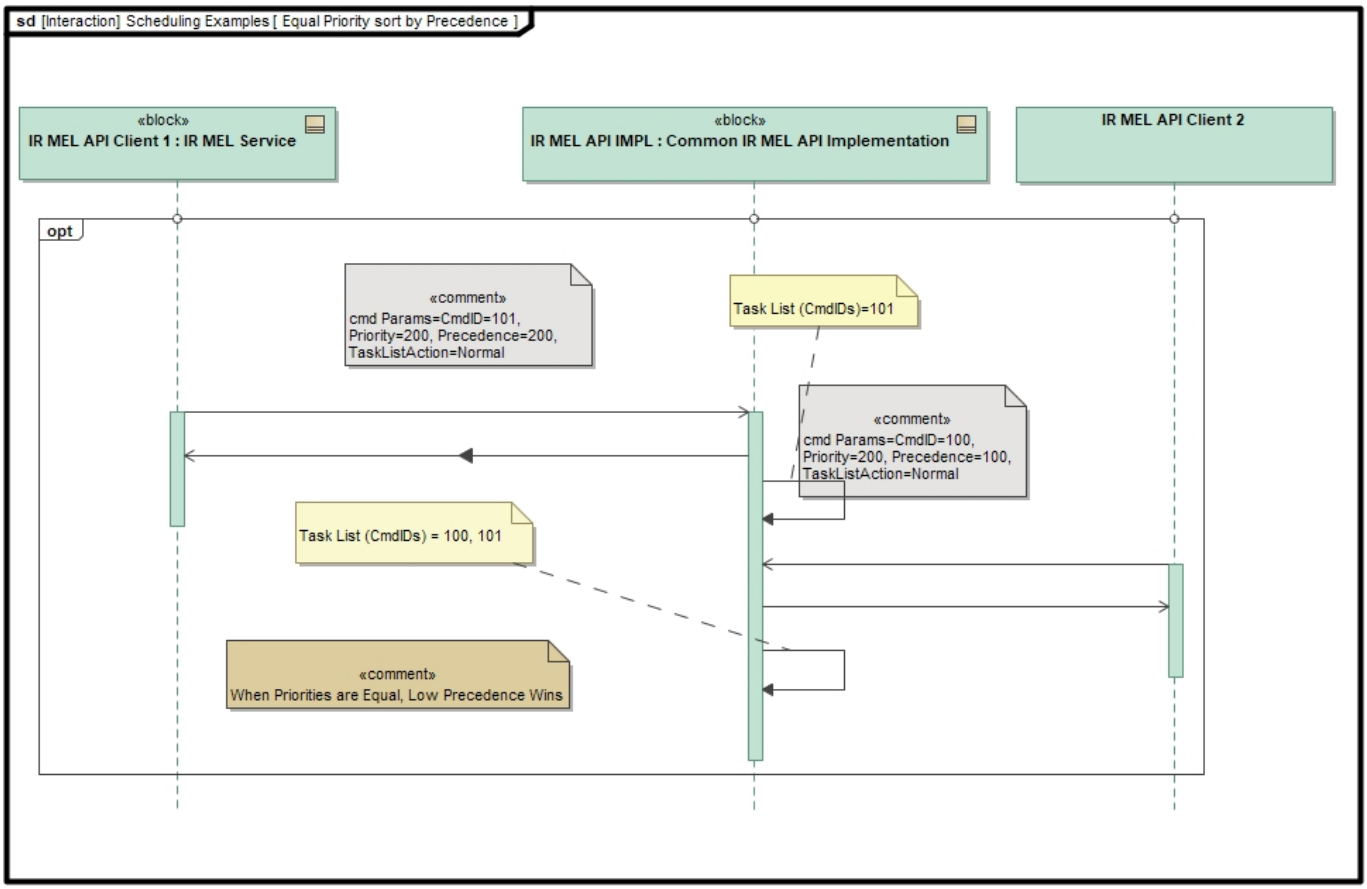


Figure 3.6.3: When Services Command Multiple Tasks of the Same Priority, the Task With the Lowest Numerical Precedence Will be Considered for Scheduling First. Precedence is Only Considered When Comparing Tasks of Equal Priority.

3.7 Command Multiple Prioritized Tasks from a Single Service

Description

The sequence diagram for this use case, shown in **Figure 3.7.1**, describes how IR MFAs may accept multiple tasks at a time from a single service, store them in the task schedule list, and execute them through the use of the `TaskListAction`.

As described in the pre-conditions, during initialization the service uses the `getCapabilities` function to query the capabilities of a particular MFA, including the `TaskScheduleDepth`. This value specifies the task list depth that the MFA supports, and is applicable to both Type 4 and Type 5 MFAs. Using `TaskScheduleDepth` gives the service control over the order in which the tasks are executed by the MFA while minimizing dead time between tasks.

A `TaskScheduleDepth` of zero means the MFA can only accept one task from any service at a time, and any new task would be immediately executed, causing the currently executing task to be aborted.

A `TaskScheduleDepth` of one means there is an executing task and an additional task can be sent to be executed once the currently executing task completes.

A `TaskScheduleDepth` of greater than one means that multiple tasks can be sent to the MFA and held in the task list. The task list is processed sequentially from front to back.

The tasks are processed based on the `TaskListAction` specified in the `sendCommand` call (NORMAL is end of list, NEXT is beginning of list, IMMEDIATE replaces executing task, ABORT ends the currently executing task, and the task at the front of the list is started immediately) so the service controls the order. If all tasks use the NORMAL priority then the list behaves like a first-in-first-out queue whereby the first task sent is executed until it completes, followed by the remaining tasks in succession.

The task schedule list allows a Type 4 MFA to be controlled like a Type 5 MFA with the service specifying the tasks and the order in which the tasks are executed.

The most basic Type 5 MFA would not need to support this mode, or could indicate it supports this mode with a `TaskScheduleDepth` of zero. More advanced Type 5 MFAs can support a `TaskScheduleDepth` greater than zero, which allows the Service to send tasks that are executed in the order specified while minimizing dead time between tasks. A Type 4 MFA could support this capability to allow a service finer control over task execution (in the case of a more sophisticated resource management service).

Preconditions

Before the MFA can accept multiple tasks at a time from a single service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning references for the C2 and Scheduling channels.
- The C2 channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `CommandStatus` from the IR MEL.
- The Scheduling channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of attached metadata listeners for `ExecuteTaskAck` and `TaskExecutingRep` from the IR MEL.
- The MFA has been commanded into the Operate state.

Postconditions

After the MFA has received and executed the tasks, the following condition should be true:

- The IR MEL is ready to receive a new task.

Assumptions

The following condition is assumed true for the Command Multiple Prioritized Tasks from a Single Service use case to be properly reported:

- The Service affirms that the MFA's `TaskScheduleDepth` is greater than or equal to two by checking relevant capabilities after configuring the IR MEL Scheduling channel.

Actors

The actors in the Command Multiple Prioritized Tasks from a Single Service use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.7.1 shows multiple tasks being sent to the IR MEL from the service:

1. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd` to command a single task with a command identification ("CmdID") of 100 and a NORMAL `TaskListAction`.

(Without fully referencing the message definition, tasks are submitted to `sendCommand` on a task list.)

2. As with any command sent to the IR MEL via `sendCommand`, the MFA provides a `CommandStatus` [over the C2 channel] in response.
3. The MFA acknowledges the receipt of task 100 with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
4. The MFA begins executing task 100.
5. The MFA indicates it is executing task 100 via the IR MEL by providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener. (The `TaskExecutingRep` provides status on the previous task as well as the currently executing task.)
6. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as task list parameters for a NORMAL `TaskListAction` task with a CmdID of 101.
7. The MFA provides a `CommandStatus` [over the C2 channel] in response.
8. The MFA acknowledges the receipt of task 101 with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
9. The MFA continues executing task 100, and places task 101 in the task list.
10. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as task list parameters for an NEXT `TaskListAction` task with a CmdID of 200.
11. The MFA provides a `CommandStatus` [over the C2 channel] in response.
12. The MFA acknowledges the receipt of task 200 with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
13. The MFA continues executing task 100. NEXT indicates that task 200 should be placed at the front of the task list, bumping 101 to the end of the list.
14. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as task list parameters for an IMMEDIATE `TaskListAction` task with a CmdID of 201.
15. The MFA provides a `CommandStatus` [over the C2 channel] in response.
16. The MFA acknowledges the receipt of task 201 with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
17. IMMEDIATE causes the currently executing task, 100, to be aborted. The MFA begins executing task 201; the task list is now {200,101}.
18. The MFA provides the `TaskExecutingRep` [over the Scheduling channel] using the attached metadata listener, indicating that task 201 is the currently executing task, and the previous task, task 100, was aborted.
19. Upon completion of task 201, the MFA executes task 200.
20. By providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener, the MFA indicates it is executing task 200 (i.e., the task at the front of the task list) and that task 201 has completed via the IR MEL.
21. Upon completion of task 200, the MFA executes task 101.

22. By providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener, the MFA indicates it is executing task 101 (i.e., the next task on the task list) and that task 200 has completed via the IR MEL.
23. Task 101 completes.
24. By providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener, the MFA indicates that it has transitioned to idle since the task list is empty, and the previous task 101 has been completed via the IR MEL.

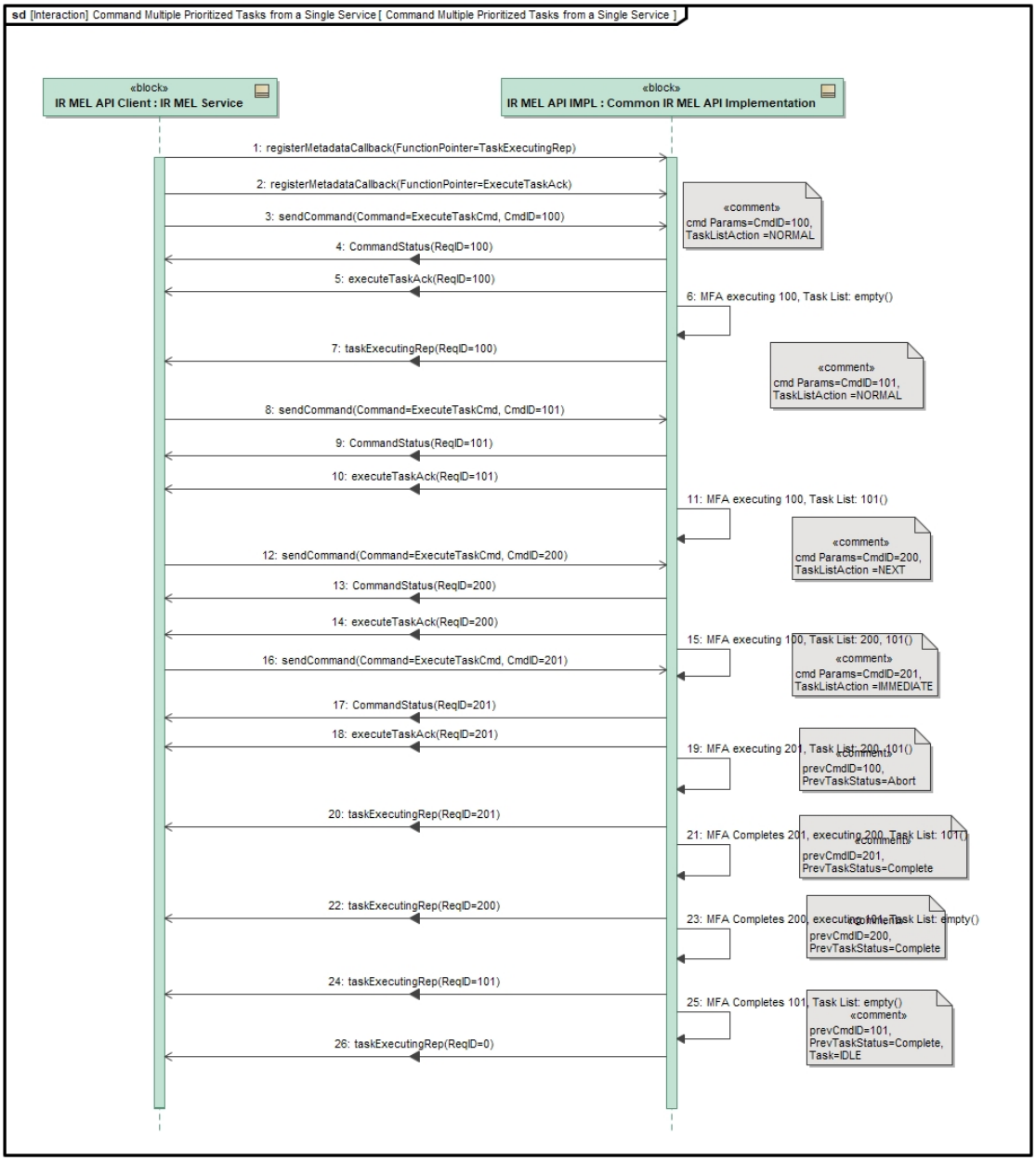


Figure 3.7.1: The Task Schedule List Allows for More Detailed Control of Task Scheduling by a Service, if Supported by the MFA.

3.8 Perform Scan

Description

The sequence diagram for this use case, shown in **Figure 3.8.1**, describes how the IR MEL provides scan support through the `ExecuteTaskCmd`.

This use case shows one service performing all of the interactions with the IR MEL for convenience. However, there may be distinct services performing different parts of this use case, such as one service monitoring the health and status of the MFA, another commanding the task, and a third collecting and processing the resultant imagery and associated metadata.

Preconditions

Before the IR MEL can provide scan support and perform all of the interactions with the service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning references for the C2, Scheduling, Health and Status, IRST Image, and Stacked Image channels.
- The C2 channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `CommandStatus` from the IR MEL.
- The Scheduling channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of attached metadata listeners for `ExecuteTaskAck` and `TaskExecutingRep` from the IR MEL.
- The HealthStatus channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `MFA_Status` from the IR MEL.
- The IRST Image channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached listener for IRST imagery from the IR MEL.
- The service affirms that the MFA is able to support a TSI stack by requesting relevant capabilities via the IR MEL Scheduling channel [using its `getCapabilities` method] and successfully performs “Configure IR MEL Channels” for the IR MEL Stacked Image channel.
- The MFA has been commanded into the Operate state.

Postconditions

After the scan has been performed, the following condition should be true:

- The MFA is ready to perform another task.

Assumptions

The following function and condition is assumed true for the scan to be properly performed:

- The service affirms that the MFA is able to perform the task by requesting relevant capabilities via the relevant IR MEL channels [using `getCapabilities` methods].

Actors

The actors in the Perform Scan use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.8.1 shows the scan being performed:

1. Periodically, the MFA sends the `MFA_Status` via the HealthStatus channel to the service, which is detected and processed by the metadata listener.
2. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as parameters required to command the scan.
3. As with any command sent to the IR MEL via sendCommand, the MFA provides a `CommandStatus` [over the C2 channel] in response.
4. The MFA acknowledges the receipt of the scan task with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
5. The MFA indicates it is executing the scan task via the IR MEL by providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener. (The `TaskExecutingRep` includes the `ActivityId` of the current task.)
6. The MFA provides LOS metadata to the service over the over the IRST Image channel, which is detected and processed by the metadata listener.

7. The MFA provides frames through the image listener method associated with the IRST Image channel. (The frame header contains metadata about the frame including `ContributingSensor` which describes the particular sensor position and type that produced the frame.)
8. If the MFA supports TSI stacks, the MFA provides stacks through the image stack listener method associated with the Stacked Image channel. (The image stack header contains metadata about the overall stack including the stack depth and image dimensions. Each image in the stack is accompanied by relevant ownship data and pointing information.)
9. When the task is complete, the IR MEL sends a `TaskExecutingRep` message indicating that the MFA has completed the task (i.e., the `ActivityId` is identically zero and the current task is idle) and the `prevActivityId` is set to the original [scan Task] `ActivityId`.

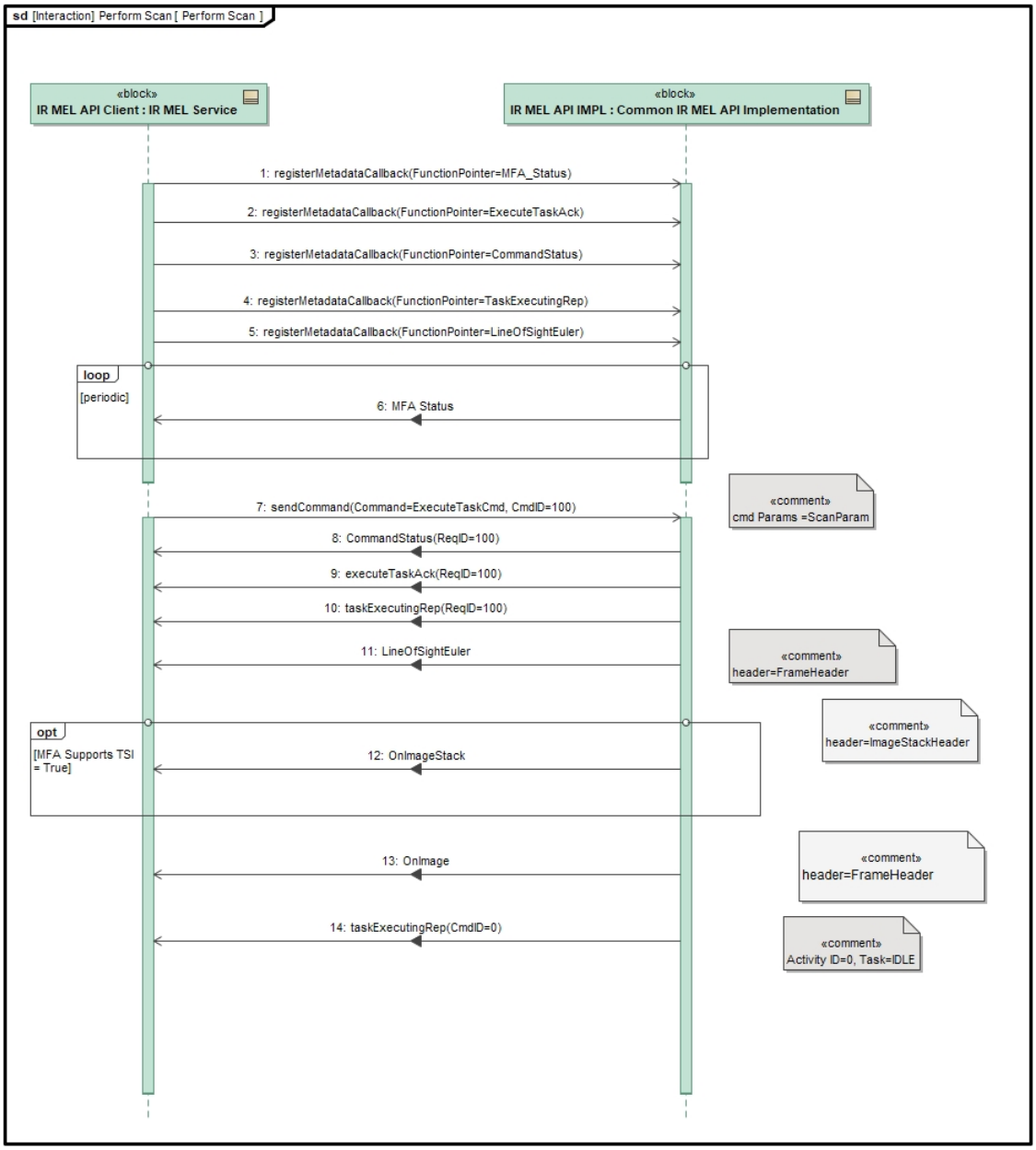


Figure 3.8.1: The IR MEL Enables Services to Command the MFA to Perform Scans. The Service Provides Scan Parameters Via the Execute Task Command.

3.9 Perform Stare

Description

The sequence diagram for this use case, shown in **Figure 3.9.1**, captures how the IR MEL provides stare support through the `ExecuteTaskCmd`.

This use case shows one service performing all of the interaction with the IR MEL for convenience. However, there may be distinct services performing different parts of this use case, such as one service monitoring the health and status of the MFA, another commanding the task, and a third collecting and processing the resultant imagery and associated metadata.

Preconditions

Before the IR MEL can provide stare support, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning references for the C2, Scheduling, Health and Status, IRST Image, and Stacked Image channels.
- The C2 channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `CommandStatus` from the IR MEL.
- The Scheduling channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of attached metadata listeners for `ExecuteTaskAck` and `TaskExecutingRep` from the IR MEL.
- The HealthStatus channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `MFA_Status` from the IR MEL.
- The IRST Image channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached listener for IRST imagery from the IR MEL.
- The service affirms that the MFA is able to support a TSI stack by requesting relevant capabilities via the IR MEL Scheduling channel [using its `getCapabilities` method] and successfully performs the Configure IR MEL Channels use case for the IR MEL Stacked Image channel.
- The MFA has been commanded into the Operate state.

Postconditions

After the stare is performed, the following condition should be true:

- The MFA is ready to perform another task.

Assumptions

The following function and condition is assumed true for the stare to be properly performed:

- The service affirms that the MFA is able to perform the task by requesting relevant capabilities via the relevant IR MEL channels [using `getCapabilities` methods].

Actors

The actors in the Perform Stare use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.9.1 shows the stare being performed:

1. Periodically, the MFA sends the `MFA_Status` via the HealthStatus channel to the service, which is detected and processed by the metadata listener.
2. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as parameters required to command the stare.
3. As with any command sent to the IR MEL via sendCommand, the MFA provides a `CommandStatus` [over the C2 channel] in response.
4. The MFA acknowledges the receipt of the stare task with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
5. The MFA indicates it is executing the stare task via the IR MEL by providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener. (The `TaskExecutingRep` includes the `ActivityId` of the current task.)
6. The MFA provides LOS metadata to the service over the over the IRST Image channel, which is detected and processed by the metadata listener.
7. The MFA provides frames through the image listener method associated with the IRST Image channel. (The frame header contains metadata about the frame including `ContributingSensor` which describes the particular sensor position and type that produced the frame.)

8. If the MFA supports TSI stacks, the MFA provides stacks through the image stack listener method associated with the Stacked Image channel. (The image stack header contains metadata about the overall stack including the stack depth and image dimensions. Each image in the stack is accompanied by relevant ownship data and pointing information.)
9. When the task is complete, the IR MEL sends a `TaskExecutingRep` message indicating that the MFA has completed the task (i.e., the `ActivityId` is identically zero and the current task is idle) and the `prevActivityId` is set to the original [stare Task] `ActivityId`.

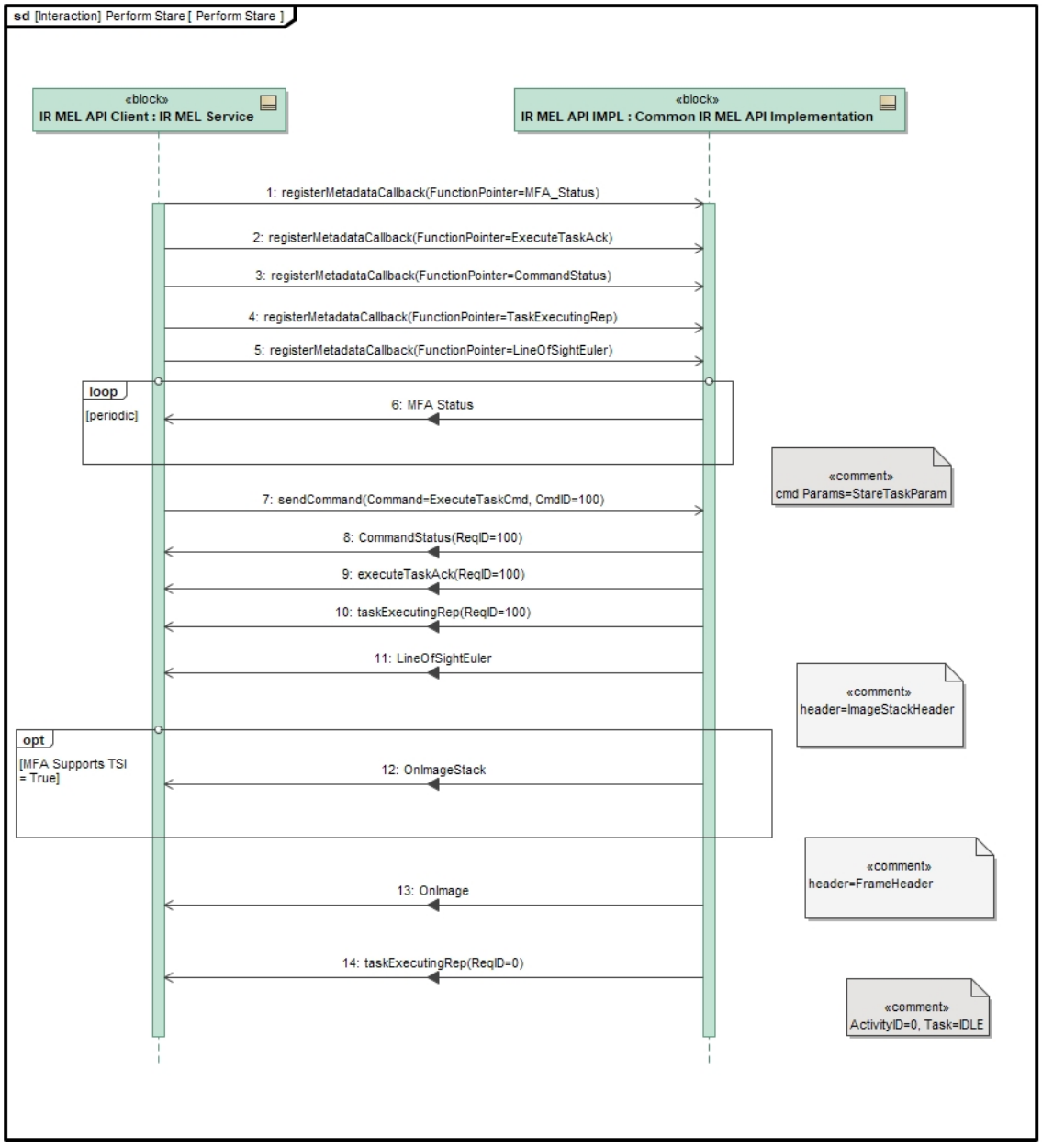


Figure 3.9.1: The IR MEL Enables Services to Command the MFA to Perform Stare. The Service Provides Stare Parameters Via the Execute Task Command.

3.10 Perform Track

Description

The sequence diagram for this use case, shown in **Figure 3.10.1**, captures how the IR MEL provides track support through the `ExecuteTaskCmd`.

This use case shows one service performing all of the interaction with the IR MEL for convenience. However, there may be distinct services performing different parts of this use case, such as one service monitoring the health and status of the MFA, another commanding the task, and a third collecting and processing the resultant imagery and associated metadata.

Preconditions

Before the IR MEL can provide track support and perform all of the interaction with the service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning references for the C2, Scheduling, Health and Status, IRST Image, and Stacked Image channels.
- The C2 channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `CommandStatus` from the IR MEL.
- The Scheduling channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of attached metadata listeners for `ExecuteTaskAck` and `TaskExecutingRep` from the IR MEL.
- The HealthStatus channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached metadata listener for `MFA_Status` from the IR MEL.
- The IRST Image channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached listener for IRST imagery from the IR MEL.
- The service affirms that the MFA is able to support a TSI stack by requesting relevant capabilities via the IR MEL Scheduling channel [using its `getCapabilities` method] and successfully performs the Configure IR MEL Channels use case for the IR MEL Stacked Image channel.
- The MFA has been commanded into the Operate state.

Postconditions

After the IR MEL has provided track support, the following condition should be true:

- The MFA is ready to perform another task.

Assumptions

The following function and condition is assumed true for the track to be properly performed:

- The service affirms that the MFA is able to perform the task by requesting capabilities via the relevant IR MEL channels [using `getCapabilities` methods].

Actors

The actors in the Perform Track use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.10.1 shows the track being performed:

1. Periodically, the MFA sends the `MFA_Status` via the HealthStatus channel to the service, which is detected and processed by the metadata listener.
2. The service uses the Scheduler channel's `sendCommand` method with a `CommandType` of `ExecuteTaskCmd`, as well as parameters required to command the track.
3. As with any command sent to the IR MEL via sendCommand, the MFA provides a `CommandStatus` [over the C2 channel] in response.
4. The MFA acknowledges the receipt of the track task with an `ExecuteTaskAck` [over the IR MEL Scheduling channel].
5. The MFA indicates it is executing the track task via the IR MEL by providing one or more `TaskExecutingRep` messages to the service [over the Scheduling channel] using the attached metadata listener. (The `TaskExecutingRep` includes the `ActivityId` of the current task.)
6. The MFA provides LOS metadata to the service over the over the IRST Image channel, which is detected and processed by the metadata listener.

7. The MFA provides frames through the image listener method associated with the IRST Image channel. (The frame header contains metadata about the frame including `ContributingSensor` which describes the particular sensor position and type that produced the frame.)
8. If the MFA supports TSI stacks, the MFA provides stacks through the image stack listener method associated with the Stacked Image channel. (The image stack header contains metadata about the overall stack including the stack depth and image dimensions. Each image in the stack is accompanied by relevant ownship data and pointing information.)
9. When the task is complete, the IR MEL sends a `TaskExecutingRep` message indicating that the MFA has completed the task (i.e., the `ActivityId` is identically zero and the current task is idle) and the `prevActivityId` is set to the original [scan Task] `ActivityId`.

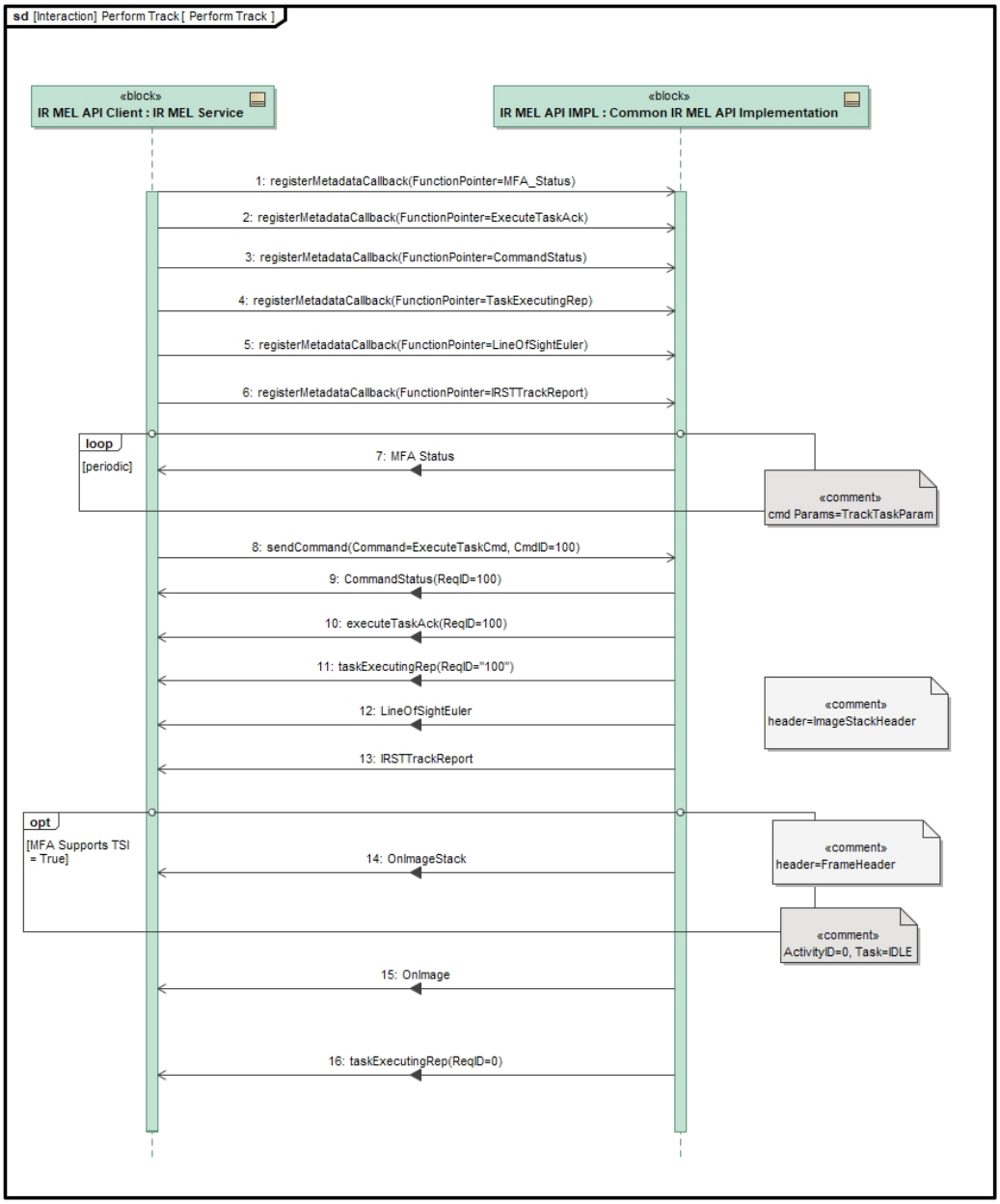


Figure 3.10.1: The IR MEL Enables Services to Command the MFA to Perform Track. The Service Provides Track Parameters Via the Execute Task Command.

3.11 Receive Detailed LOS Track

Description

A `LOS3D_KinematicsType` data object is an object that the MFA has detected in the image frame, and reports to the service as a potential target to continue observing or tracking. The `LOS3D_KinematicsType` metadata provided through the IR MEL consists of the relative position, relative angles including uncertainty, relative slant range including uncertainty, and covariance. As shown in **Figure 3.11.1**, the IR MEL provides `LOS3D_KinematicsType` metadata through the use of the metadata pipe using the `LOS3D_KinematicsType`. An IR MEL implementation provides the `LOS3D_KinematicsType` as they are generated by the MFA. There is not a command required from the service to prompt the generation of this metadata.

Preconditions

Before the MFA can detect the object in the image frame and report it to the service, the following conditions must be true:

- The service has successfully performed the Initialize IR MEL for Use use case, returning a reference to the IRST Image channel.
- The IRST Image channel has been set up using the Configure IR MEL Channels use case, resulting in the instantiation of an attached listener for IRST imagery as well as an attached metadata listener for `LOS3D_KinematicsType` from the IR MEL.
- The MFA has been commanded into the Operate state.

Postconditions

After the `LOS3D_KinematicsType` data has been detected and reported to the service, the following condition should be true:

- The MFA is ready to perform another task.

Assumptions

The following functions and conditions are assumed to be true for a detailed `LOS3D_KinematicsType` data to be properly reported:

- The service affirms that the MFA is able to support the Detailed LOS Track data by requesting relevant capabilities via the relevant IR MEL channels [using `getCapabilities` methods].

Actors

The actors in the Receive Detailed LOS Track use case are:

- The service
- MEL API implementation

Flow of Events

Figure 3.11.1 shows the `LOS3D_KinematicsType` data object being detected by the MFA and reported to the service:

1. When the MFA detects an object, it provides detailed kinematic track data to the service over the IRST Image channel.
2. The Detailed LOS Track is detected and processed by `LOS3D_KinematicsType` metadata listener established through the IRST Image channel.

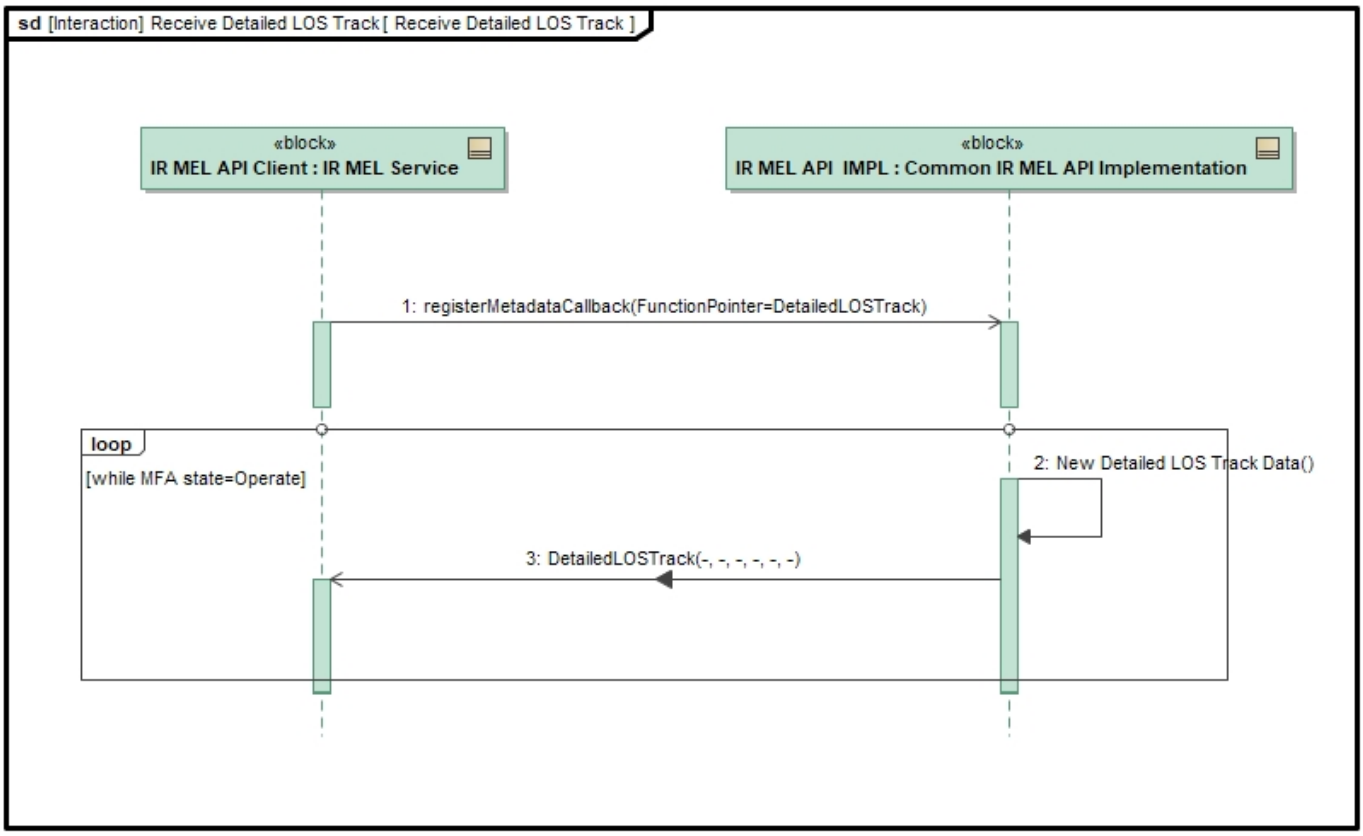


Figure 3.11.1: Once a Service Registers the Metadata Callback for `DetailedLOSTrack`*, the MFA Will Report Potential Target Information Back to the Service Without Needing a Command.*

4.0 IR MEL Support Functions

This chapter describes functions provided by the IR MEL which are not driving use cases, but support the primary functionality of the IR MEL.

4.1 Channel Comms Test

The Channel Comms Test support function, in **Figure 4.1.1**, illustrates a method to test the communications path between a service that has enabled a channel and the IR MEL. The service can send the test request, the IR MEL communicates with the MFA, and the IR MEL responds with the acknowledgment, which confirms to the service that the communications path to the IR MEL on that channel is functional.

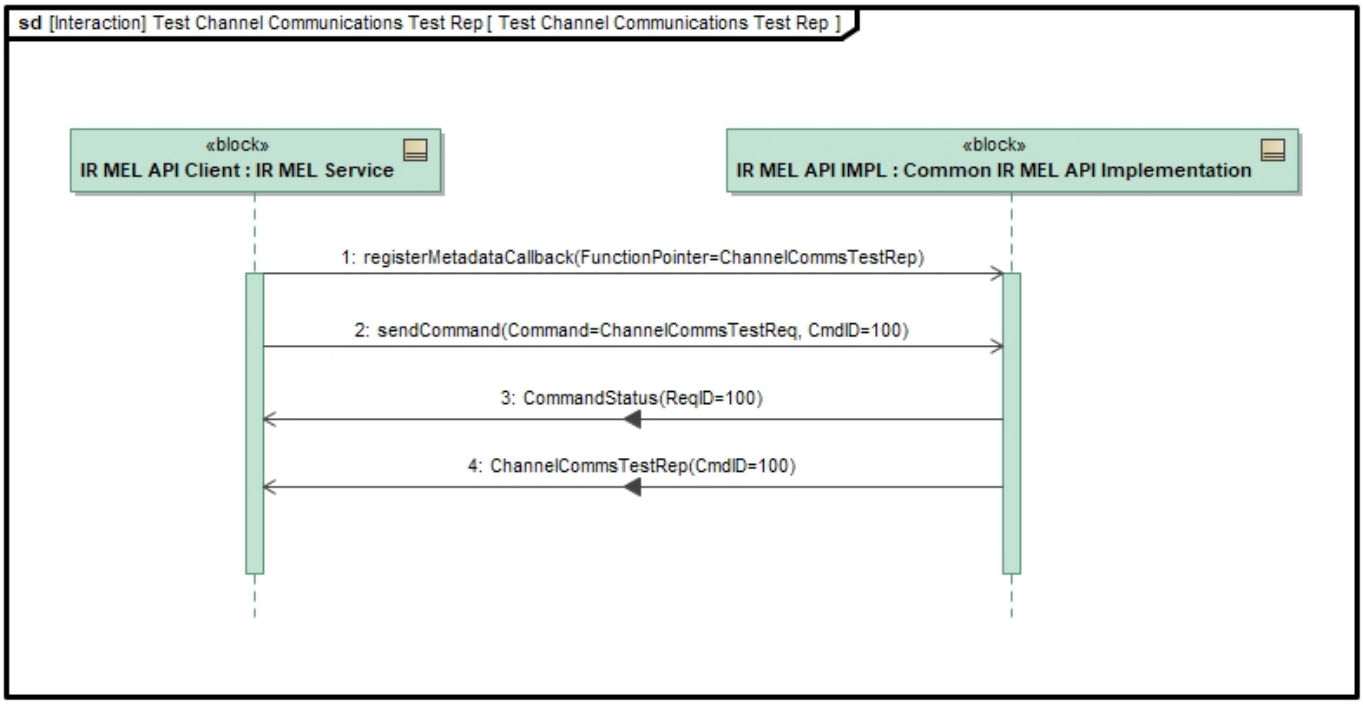


Figure 4.1.1: MFA to Service Communications Occur Over Channels, Which May be Verified With a Channel Comms Test.

4.2 Instrumentation

The instrumentation support function, in **Figure 4.2.1**, is provided to allow the MFA to pass data through the IR MEL to a Service that may wish to record instrumentation data, using (e.g.) an OMS adopter. This data may be used for mission planning and analysis, debugging, or mission playback and forensics. The instrumentation channel is set up analogously to the imagery channels by providing buffer management functionality. The service registers the buffer and enables the channel to receive data using an instrumented data listener. The service has the option, through the `InstrumentationLevelCmd`, to set the priority of the data to either "normal" or "debug". The IR MEL responds with the `InstrumentationRep` indicating the new priority. The MFA provider is responsible for controlling the data output of each priority.

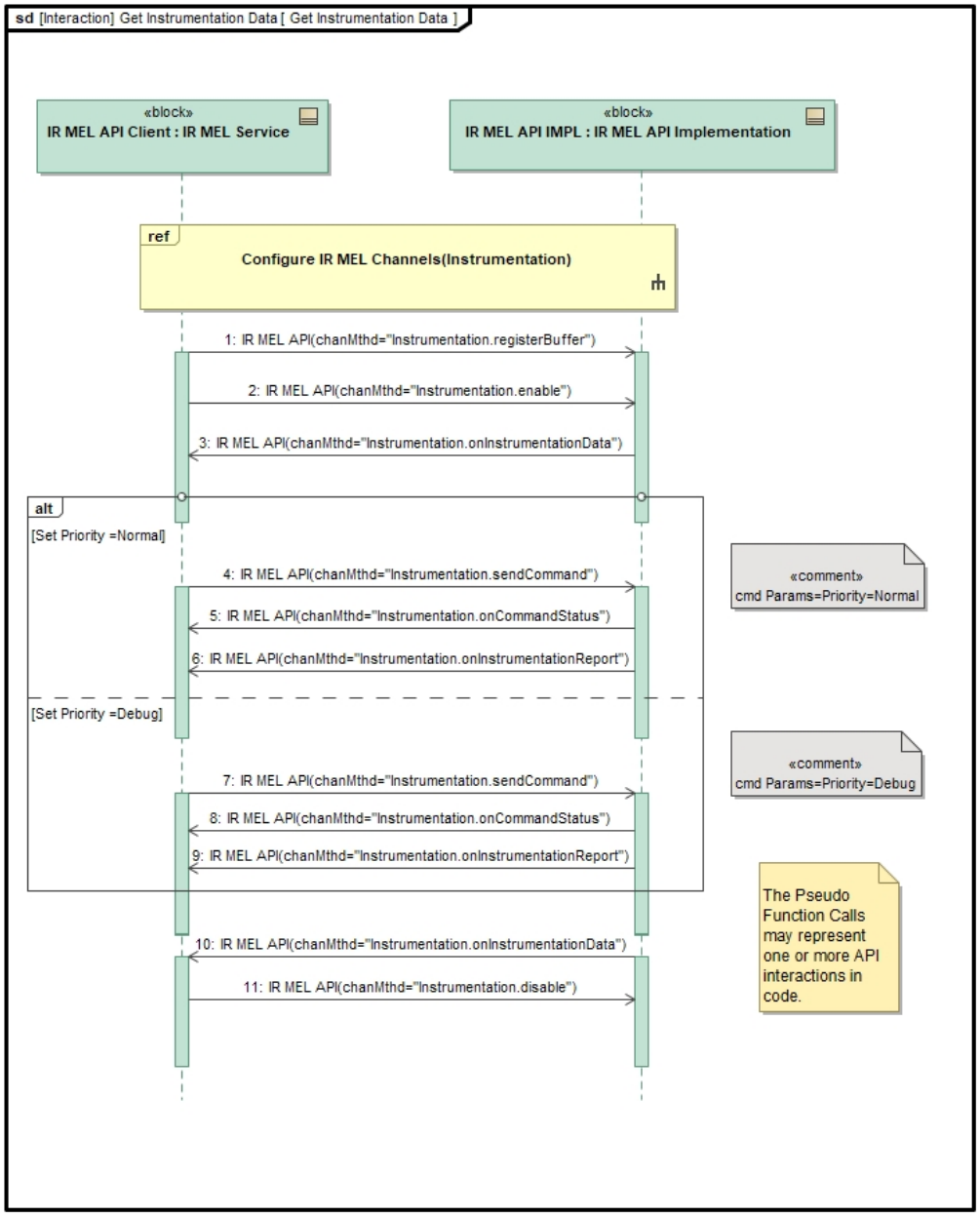


Figure 4.2.1: The IR MEL Provides a Dedicated Channel for Instrumentation Data, Which May be Used for Analysis, Debugging, or Mission Playback.

4.3 Navigation Report

The navigation report, in **Figure 4.3.1**, is sent from the service to the MFA to provide navigation data such as latitude, longitude, altitude, speed, and acceleration.

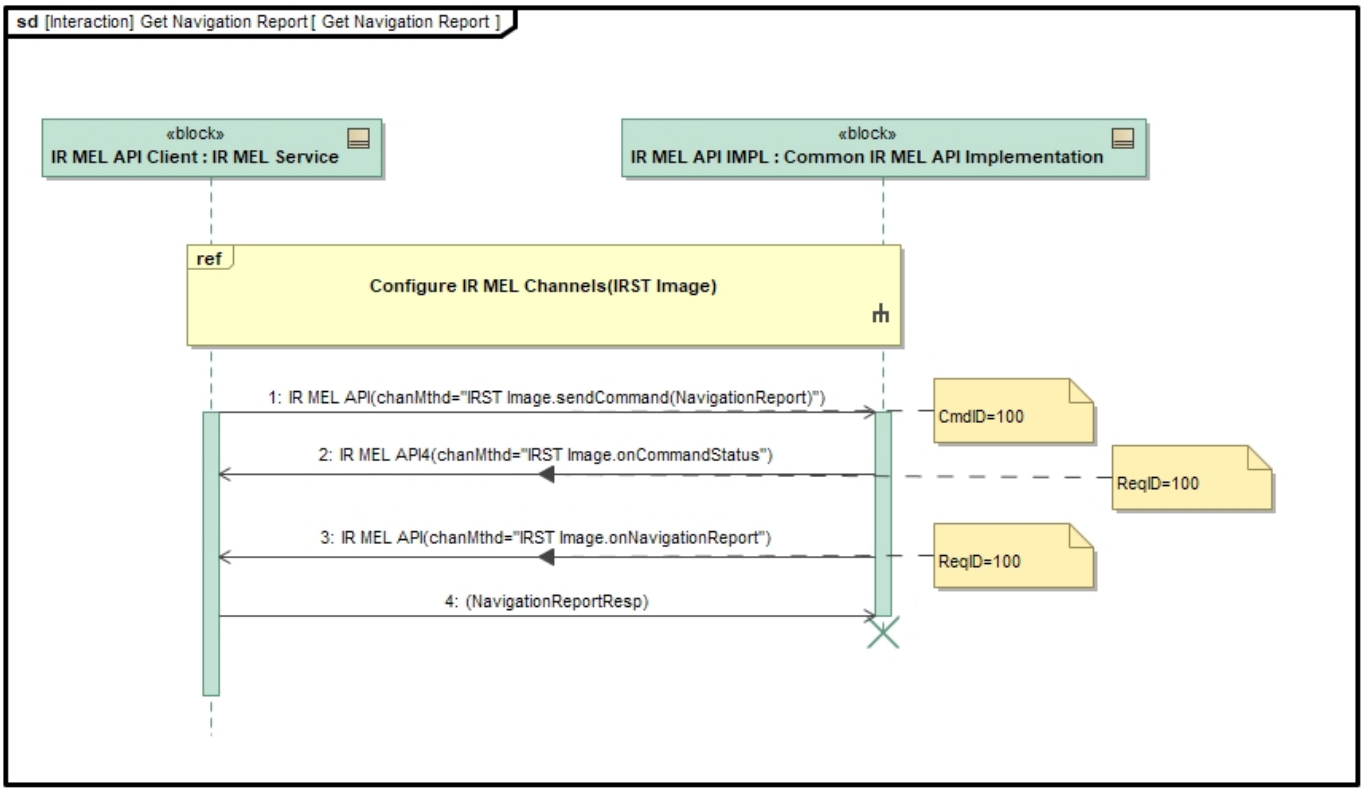


Figure 4.3.1: Navigation Data Can be Provided to the IR MFA By a Service Via the Navigation Report.

4.4 Keepalive

The `KeepaliveRep` command, in **Figure 4.4.1**, is used for the service to inform the MFA that the service is still connected and operating when there are no other commands for the MFA. The `cmd` field in `onCommand` is blank for the `KeepaliveRep`.

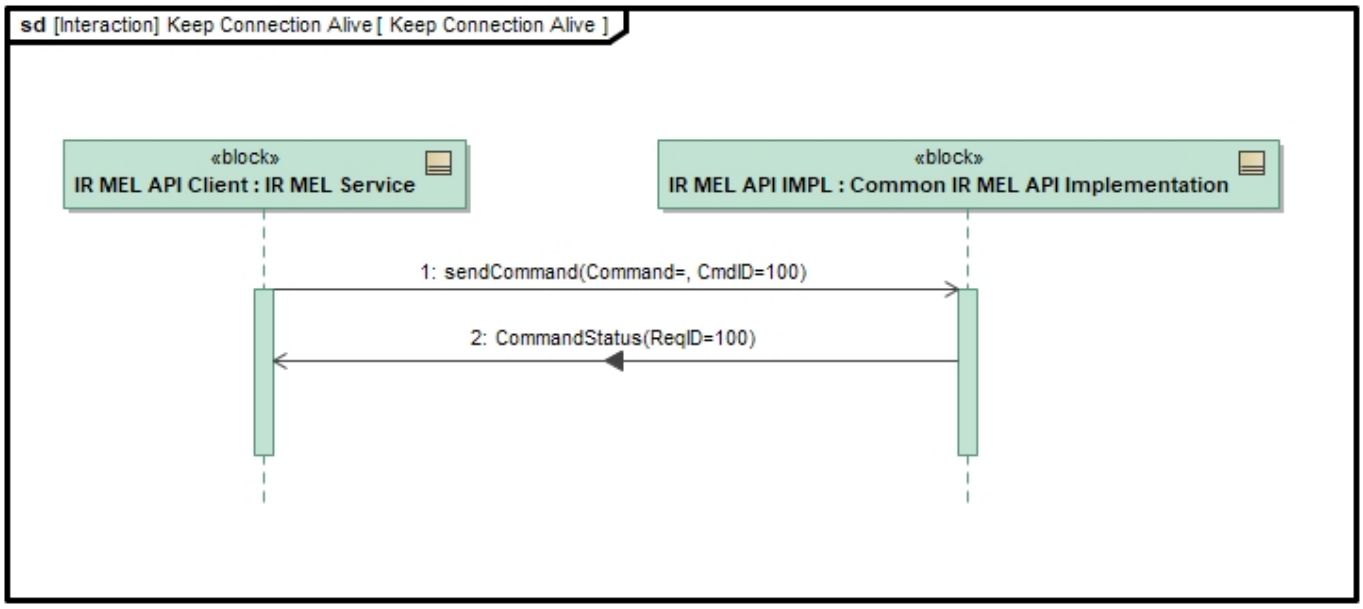


Figure 4.4.1: A Service Can Inform the MFA That it is Still Connected and Active by Sending an Empty Command.

4.5 Get Track Data

When a service initially attaches to a channel, as in **Figure 4.5.1**, it provides the `Config` class, which contains information about the channel and platform. `Config` also contains a variable `supportsSystemTrackData`. This boolean variable informs the MFA whether or not the service can provide track data. The MFA can request track data from the service through the `RequestSystemTrackData`. The service responds with `SystemTrackDataResponse`.

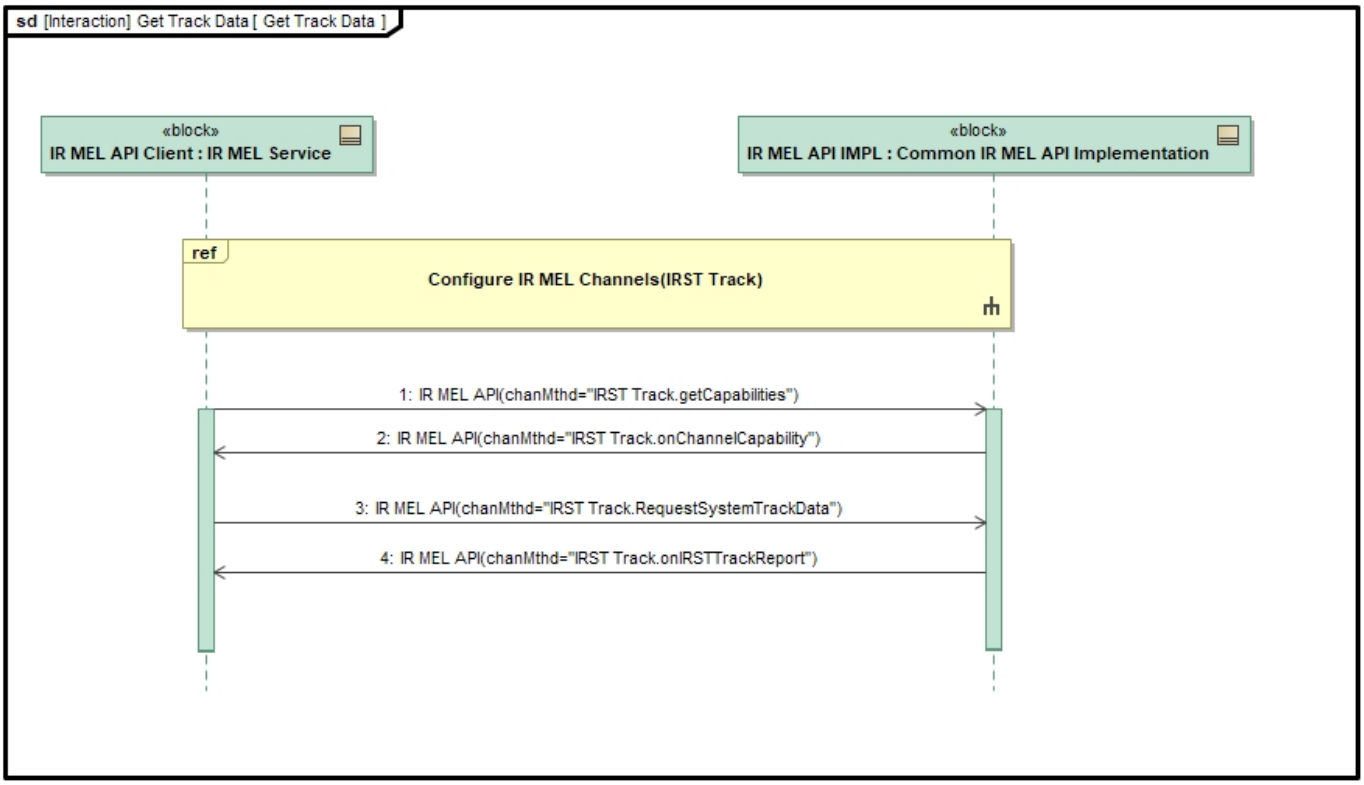


Figure 4.5.1: The MFA Can Prompt the Service to Provide Track Data Using the Request System Track Data Metadata Callback.

4.6 Track Data Update

When a service initially attaches to a channel, as in **Figure 4.6.1**, it provides the `Config` structure, which contains information about the channel and platform. `Config` also contains a variable `supportsTrackDataUpdates`. This boolean variable informs the MFA whether or not the service can push track data updates. The service can create, update, or delete track data from the MFA through the `TrackDataUpdate` command type. The MFA responds with `CommandStatus`.

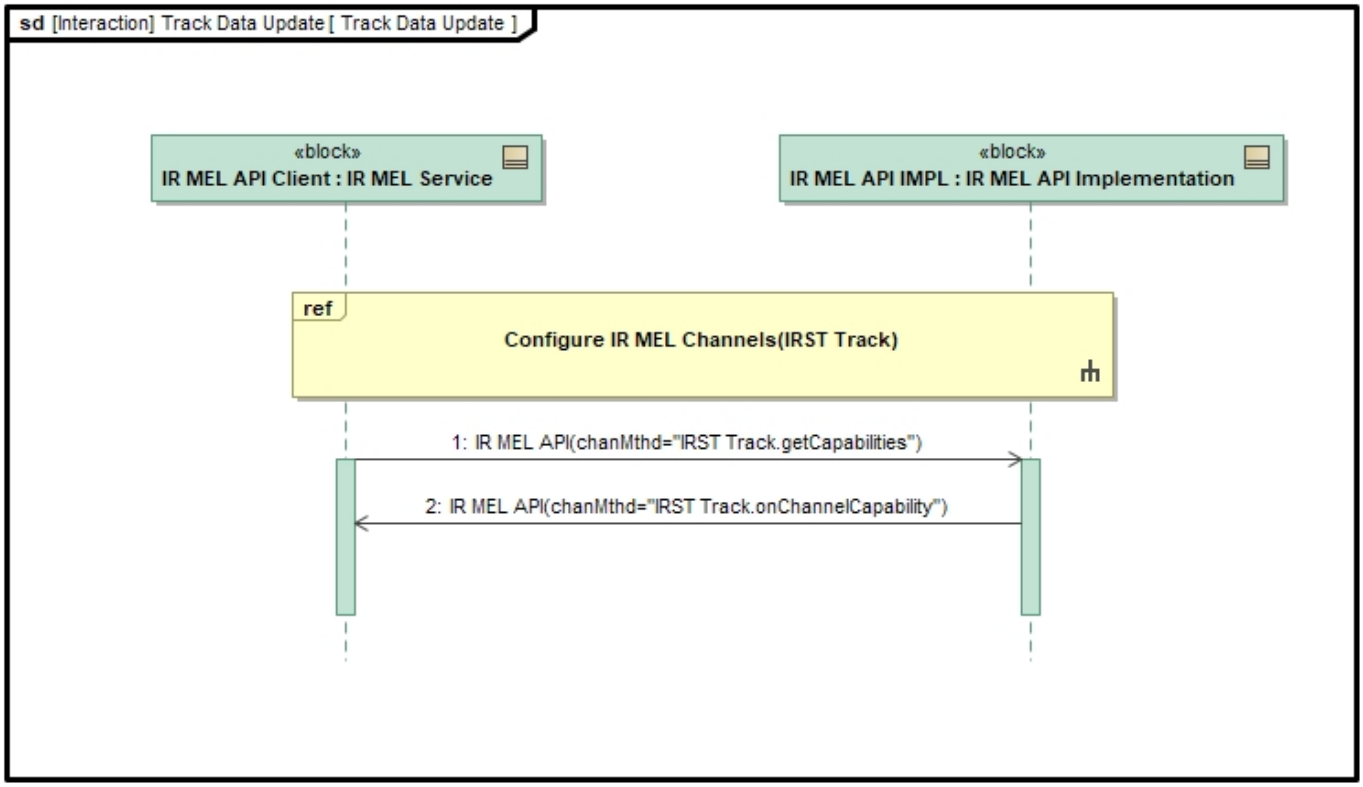


Figure 4.6.1: If the Service Supports Track Data Updates to the MFA, Then the Service Can Create, Update, or Delete Track Data With the `TrackDataUpdate` *Command.*

4.7 Camera Protect

When a service initially attaches to a channel, as in **Figure 4.7.1**, it provides the `Config` structure, which contains information about the channel and platform. `Config` also contains the variable `supportsCameraProtect`. This boolean variable informs the MFA whether or not the service can send a camera protect command. The service provides a protect flag and a response time value to the MFA through the `CameraProtect` command type. The MFA responds with `CommandStatus` and a `CameraProtectCmdResp` which should return an `OK`, `Unavailable`, or `Unsupported`.

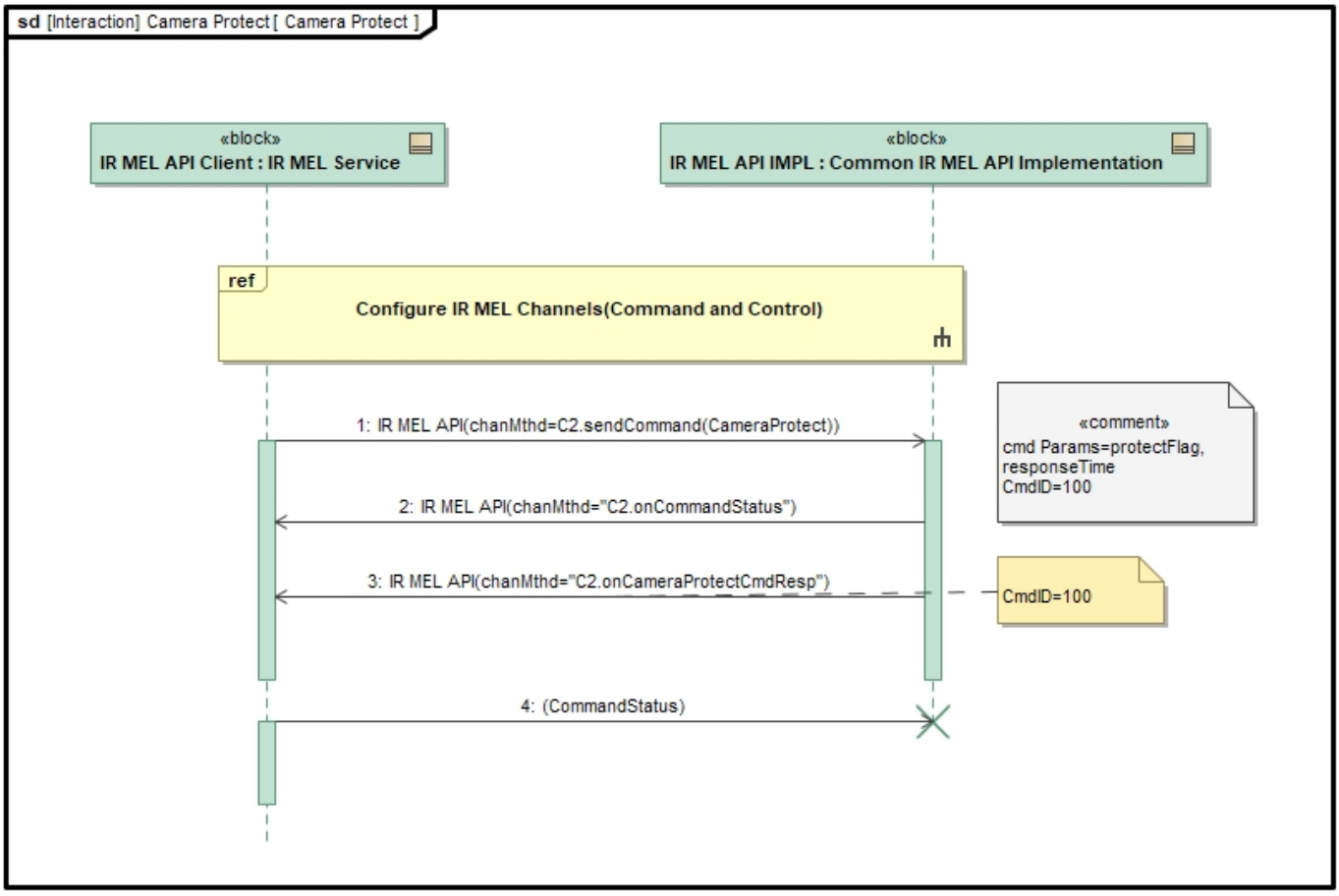


Figure 4.7.1: When a Service Issues a Camera Protect Command, the MFA Will Respond With Its Ability to Comply.

4.8 Zeroize

Zeroize is the process of indicating a command to erase cryptographic data and associated classified data in the subsystem. The Mission System Integrator (MSI) can choose any mechanism to zeroize. The MSI may not be free to choose any mechanism depending on the requirements imposed on them. The mechanisms chosen by the MSI may include special signals, security discrete, or MEL calls. If the MSI chooses security discrete or special signals, the MEL implementation `Zeroize` call will be present but may not actually perform zeroization.

5.0 IR MEL Channels

As described in **Chapter 2**, services create one or more channels to an IR MFA and perform all command and control of the MFA through these channels. Services also receive metadata and imagery data through the IR MEL channels.

There are seven IR MEL channels (shown in **Figure 5.0.1**):

1. Command and Control
2. Health and Status
3. Scheduling
4. Instrumentation
5. IRST Image
6. IRST Track
7. Stacked Image

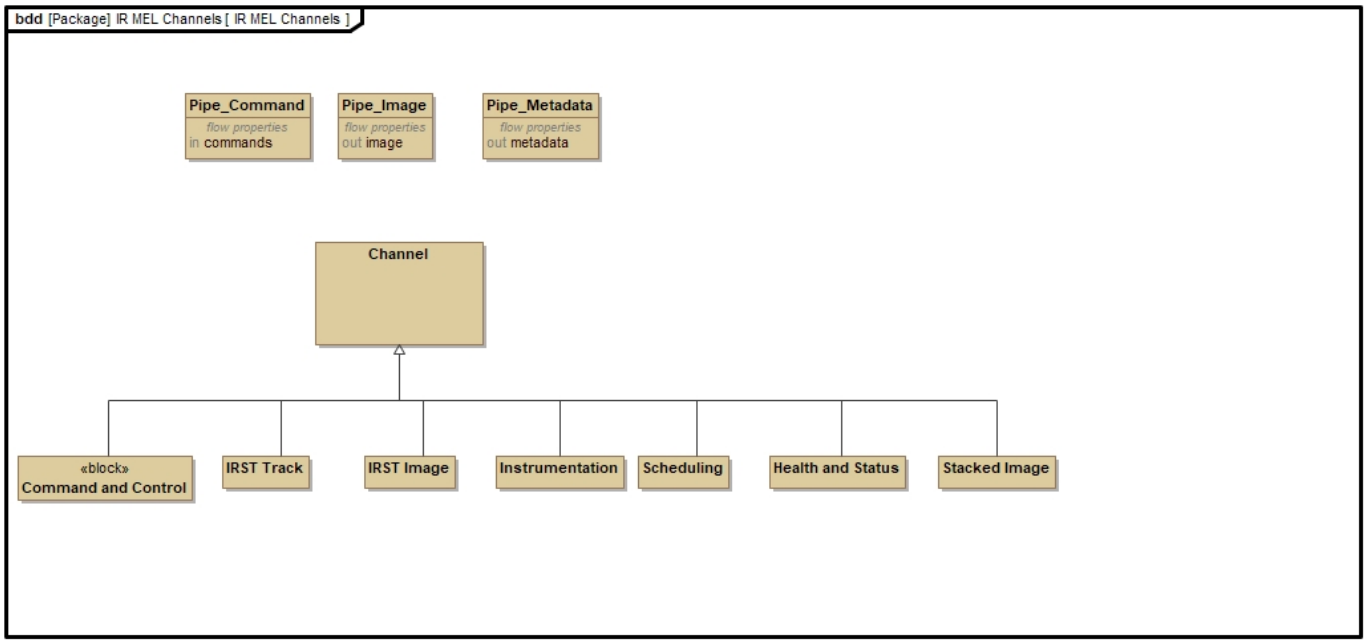


Figure 5.0.1: The IR MEL Supports Seven Different Types of Communication Channels. Each Channel may Allow For the Transfer of Commands, Image Data, and Metadata.

The service should only connect to the channel(s) required for its desired interactions with the IR MEL. The commands and metadata exchanged across each channel are described in the following sections.

5.1 Command and Control

Description

The Command and Control channel is used to command features such as BIT status, states, and modes. Commands are sent to the IR MEL on this channel in the form of `CommandTypes`, listed below. The data that is sent back from the IR MEL in response to these commands are defined by the metadata types listed below.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by the `CommandAndControl` channel and can be used by the service to test the communications path to the IR MEL on this channel.
- The `Camera` command is used to set up parameters for the camera such as the frame rate.
- The `BIT` command is used to initiate BITs, cancel BITs, or clear faults.
- The `SetConfig` command is used to set up any configurations of the MFA.
- The `ModeCmd` command is used to change the state and mode of the MFA.
- The `SystemTrackDataResponse` is used to send track data stored in the service to the MFA.
- The `TrackDataUpdate` command allows the service to create, update, or delete track data from the MFA.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working as expected.
- `BIT_Configuration` is used to provide information about all of the components for which the MFA can report BIT.
- `CommandStatus` is used to report status on a command or why the MFA cannot comply.
- `BIT_Status` is used to report BIT for all the components of the MFA.

5.2 Health and Status

Description

The Health and Status channel is used as a consumer only channel to receive health and status and BIT data.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq`, in order to confirm that the communications path between the service and IR MEL is working as expected.
- `BIT_Status` is used to report BIT for all the components of the MFA.
- `MFA_Status` is used to report the current MFA state and mode and the status of each MFA component.
- `LF_Status` is used to report the current status of a local function on the MFA.

5.3 Scheduling

Description

The Scheduling channel is used for the service to request tasks for the MFA to perform now or in the future.

Commands

The `CommandTypes` supported by this channel are described below.

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.

- The `ExecuteTaskCmd` command is used to command individual tasks with associated priorities.
- The `PrioritizedTrackList` command is used to provide the MFA with prioritized targets to track. This command is only applicable for MFAs that support Scan Volume Schedule. The MFA creates a unified schedule based on the prioritized track input from all connected services.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working as expected.
- `ExecuteTaskAck` is used to acknowledge that the IR MEL received the task from the service.
- `TaskExecutingRep` is used to provide status on the currently executing task and previous task.

5.4 Instrumentation

Description

The Instrumentation channel is used for the MFA to provide debug and mission replay data to a service to be recorded and logged for later use.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.
- The `InstrumentationLevelCmd` command is used to command the level of data that is provided by the MFA with the options being debug or normal.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working

as expected.
- `InstrumentationReport` is used to provide data to be logged from the MFA.

5.5 Infrared Search and Track (IRST) Image

Description

The IRST Image channel is used to provide IRST imagery and metadata.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.
- The `Camera` command is used to set up parameters for the camera such as the frame rate.
- The `NavigationReport` command is used to request the navigation data from the MFA.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working as expected.
- `BadPixelList` is used to provide information about pixels that have some defect which processing could mitigate.
- `OpticalDistortionMap` is used to provide details on the optical distortion map of the MFA.
- `LineOfSightReport` is one of two required metadata types to provide LOS data from the MFA.
- The `LineOfSightEuler` is one of two required metadata types to provide LOS data from the MFA.
- `LineOfSightQuaternion` is an optional metadata type to provide LOS data from the MFA.
- `CandidateObjectMessage` is used for the MFA to report that it has detected some objects which may be candidates to further track, take data on, or process. If the candidate objects contain additional information populated by preprocessing internal to the MFA, `CandidateObjectPreProc` may be used in place of `CandidateObject` within the `CandidateObjectMessage` .

5.6 IRST Track

Description

The IRST Track channel is used to command and provide data on targets being tracked.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.
- The `SystemTrackDataResponse` is used to send track data stored in the service to the MFA.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working as expected.
- `IRSTTrackReport` is used to provide information about targets being tracked by the MFA.
- `CandidateObjectMessage` is used by the MFA to report that it has detected some objects which may be candidates to further track, take data on, or process. If the candidate objects contain additional information populated by preprocessing internal to the MFA, `CandidateObjectPreProc` may be used in place of `CandidateObject` within the `CandidateObjectMessage`.
- `RequestSystemTrackData` is used by the MFA to request track data that is stored in the service.

5.7 Stacked Image

Description

The Stacked Image channel is used to provide IRST TSI imagery and metadata.

Commands

The `CommandTypes` supported by this channel are:

- The `KeepaliveRep` is supported by all channels and can be used by the service as a heartbeat.
- The `ChannelCommsTestReq` is supported by all channels and can be used by the service to test the communications path to the IR MEL on this channel.
- The `ConfigSetCommand` is supported by all channels and can be used to set up any configurations of the MFA.

Metadata

The metadata types supported by this channel are:

- `ChannelCommsTestReport` is supported by all channels. It is sent by the IR MEL in response to the `ChannelCommsTestReq` to confirm that the communications path between the service and IR MEL is working as expected.
- The `ConfigSetCommand` is used to set additional channel-specific configurations on the MFA. For the Stacked Image channel, this is used to set the `StackDepth`. The `StackDepth` is the number of frames in an individual TSI, from 1 to 11 inclusive.

A.0 Acronyms

ABB	Avionics Backplane Bus
AMS	Agile Mission Suite
API	Application Programming Interface
ASB	Abstract Service Bus
BIT	Built-In Test
C2	Command and Control
GRA	Government Reference Architecture
ID	Identification
IDD	Interface Description Document
IR	Infrared
IRST	Infrared Search and Track
LCC	Line of Sight Coordinate Correction
LOS	Line of Sight
MEL	MFA Encapsulation Layer
MFA	Multi-Function Aperture
MFP	Multi-Function Processor
MPS	Mission Processing Subsystem
MPU	Minimum Procurable Unit
MSI	Mission System Integrator
OMS	Open Mission Systems
RF	Radio Frequency
SK	Starter Kit
TSI	Temporally Spaced Image
UCI	Universal Command and Control Interface
UUID	Universally Unique Identifier

B.0 Revision History

The Revision History table, **Table B.0.1**, captures major changes made for a given AMS GRA volume as the SK continues to mature.

Revision	Description	Date	Notes
-	This revision adds a revision table to the volume.	May 22,2024	SK11.0
A	This revision adds prose updates and clarifications.	July 24, 2025	SK13.0
B	The IR MEL code was updated with several minor improvements including bug fixes and the addition of navigation data in the frame header.	April 24, 2026	AMS GRA 14.0

Table B.0.1: Revision History Captures the Major Changes Made to the IR MEL IDD Between Documentation Releases.